20EIE507T - MODEL BASED DEVELOPMENT OF CYBER-PHYSICAL SYSTEMS

Dr.G.Y.Rajaa Vikhram,
Assistant Professor,
Dept. of EIE

Hybrid Systems & State Machines
9
Modal Models – Combining Discrete and Continuous Dynamics
Actor Model for State Machines
Actor representation of FSM
Continuous Inputs
Thermostat example
State Refinements, Notations of Hybrid Systems, Classes of Hybrid systems
Tutorial
Timed Automata
Higher Order Dynamics

Timed automation variant of traffic light controller
Hybrid system model for mass system
Supervisory Control
Automated guided vehicle, Composition of state machines, Concurrent composition
Tutorial
Side-by-Side Synchronous Composition
Side-by-Side Asynchronous Composition
Shared Variables
Cascade Composition
General Composition
Hierarchical State Machines
Tutorial

Hybrid Systems

Chapters 1 and 2 describe two very different modeling strategies, one focused on continuous dynamics and one on discrete dynamics. For continuous dynamics, we use differential equations and their corresponding actor models. For discrete dynamics, we use state machines.

Cyber-physical systems integrate physical dynamics and computational systems, so they commonly combine both discrete and continuous dynamics. In this chapter, we show that the modeling techniques of Chapters 1 and 2 can be combined, yielding what are known as **hybrid systems**. Hybrid system models are often much simpler and more understandable than brute-force models that constrain themselves to only one of the two styles in Chapters 1 and 2. They are a powerful tool for understanding real-world systems.

4.1 Modal Models

In this section, we show that state machines can be generalized to admit continuous inputs and outputs and to combine discrete and continuous dynamics.

4.1.1 Actor Model for State Machines

In Section 3.3.1 we explain that state machines have inputs defined by the set *Inputs* that may be pure signals or may carry a value. In either case, the state machine has a number of input ports, which in the case of pure signals are either present or absent, and in the case of valued signals have a value at each reaction of the state machine.

We also explain in Section 3.3.1 that actions on transitions set the values of outputs. The outputs can also be represented by ports, and again the ports can carry pure signals or valued signals. In the case of pure signals, a transition that is taken specifies whether the output is present or absent, and in the case of valued signals, it assigns a value or asserts that the signal is absent. Outputs are presumed to be absent between transitions.

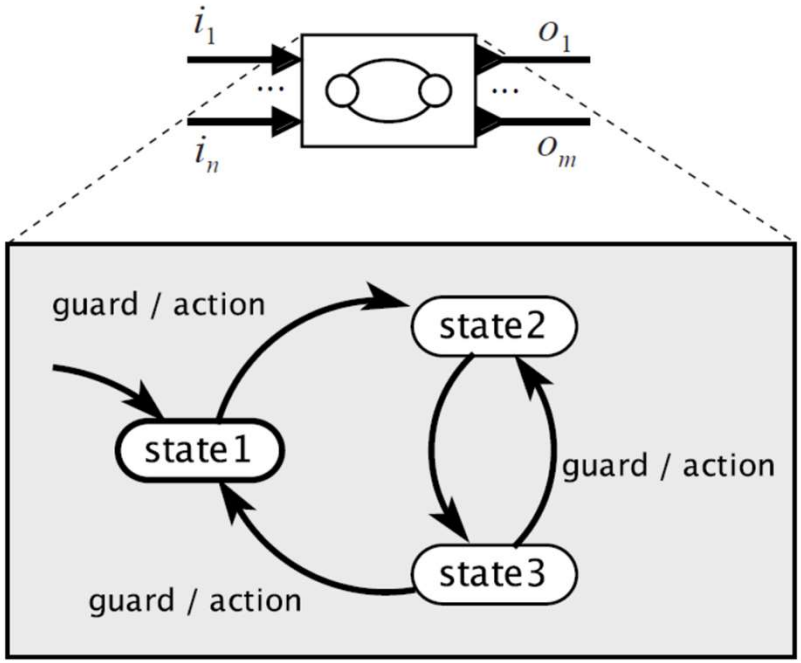

Figure 4.1: An FSM represented as an actor.

Given this input/output view of state machines, it is natural to think of a state machine as an actor, as illustrated in Figure 4.1. In that figure, we assume some number n of input ports named $i_1 \cdots i_n$. At each reaction, these ports have a value that is either *present* or *absent* (if the port carries a pure signal) or a member of some set of values (if the port carries a valued signal). The outputs are similar. The guards on the transitions define subsets of possible values on input ports, and the actions assign values to output ports. Given such an actor model, it is straightforward to generalize FSMs to admit continuous-time signals as inputs.

4.1.2 Continuous Inputs

We have so far assumed that state machines operate in a sequence of discrete reactions. We have assumed that inputs and outputs are absent between reactions. We will now generalize this to allow inputs and outputs to be continuous-time signals.

In order to get state machine models to coexist with time-based models, we need to interpret state transitions to occur on the same timeline used for the time-based portion of the system. The notion of discrete reactions described in Section 3.1 suffices for this purpose, but we will no longer require inputs and outputs to be absent between reactions. Instead, we will define a transition to occur when a guard on an outgoing transition from the current state becomes enabled. As before, during the time between reactions, a state machine is understood to be stuttering. But the inputs and outputs are no longer required to be absent during that time.

Example 4.1: Consider a thermostat modeled as a state machine with states $\Sigma = \{\mathsf{heating}, \mathsf{cooling}\}$, shown in Figure 4.2. This is a variant of the model of Example 3.5 where instead of a discrete input that provides a temperature at each reaction, the input is a continuous-time signal $\tau\colon \mathbb{R} \to \mathbb{R}$ where $\tau(t)$ represents the temperature at time t. The initial state is cooling, and the transition out of this state is enabled at the earliest time t after the start time when $\tau(t) \leq 18$. In this example, we assume the outputs are pure signals *heatOn* and *heatOff*.

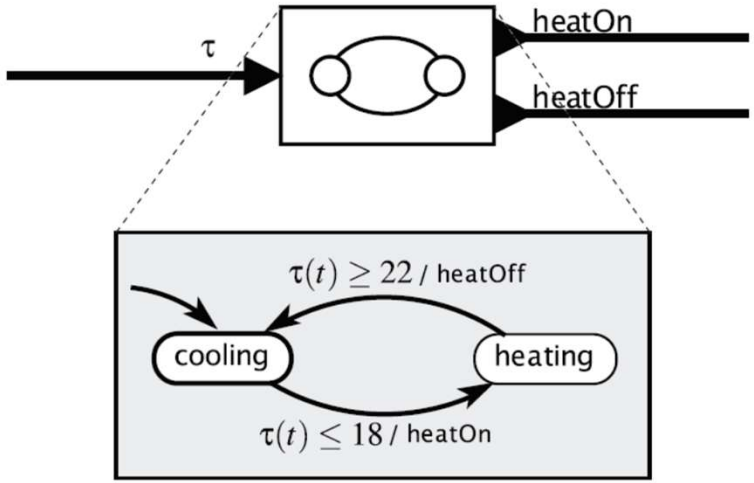

Figure 4.2: A thermostat modeled as an FSM with a continuous-time input signal.

In the above example, the outputs are present only at the times the transitions are taken. We can also generalize FSMs to support continuous-time outputs, but to do this, we need the notion of state refinements.

4.1.3 State Refinements

A hybrid system associates with each state of an FSM a dynamic behavior. Our first (very simple) example uses this capability merely to produce continuous-time outputs.

Example 4.2: Suppose that instead of discrete outputs as in Example 4.1 we wish to produce a control signal whose value is 1 when the heat is on and 0 when the heat is off. Such a control signal could directly drive a heater. The thermostat in Figure 4.3 does this. In that figure, each state has a refinement that gives the value of the output h while the state machine is in that state.

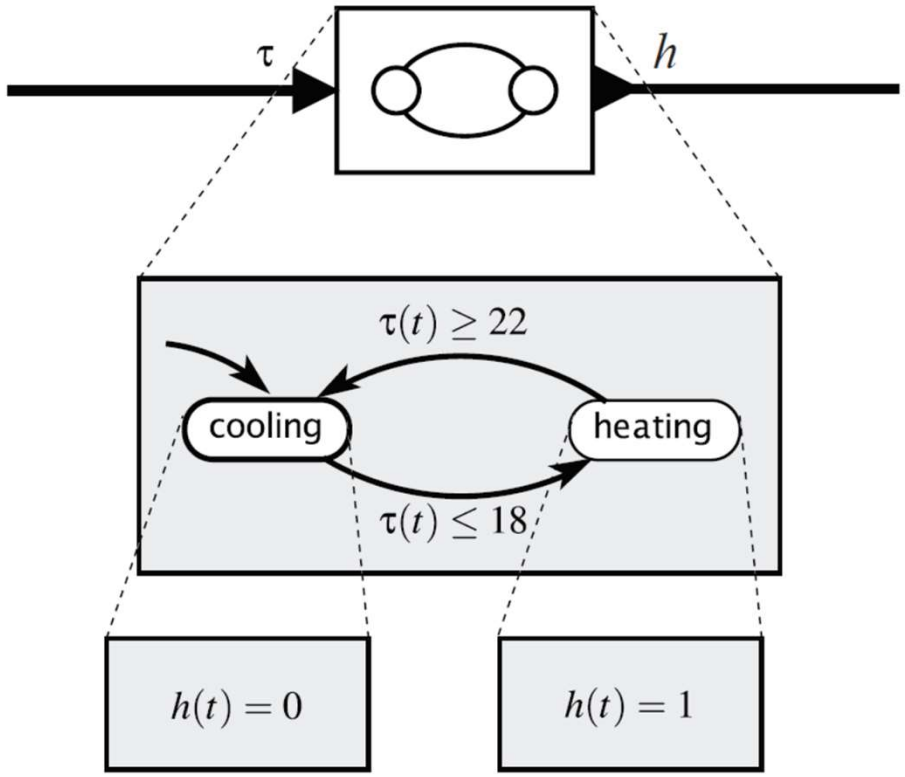

Figure 4.3: A thermostat with continuous-time output.

In a hybrid system, the current state of the state machine has a **state refinement** that gives the dynamic behavior of the output as a function of the input. In the above simple example, the output is constant in each state, which is rather trivial dynamics. Hybrid systems can get much more elaborate.

The general structure of a hybrid system model is shown in Figure 4.4. In that figure, there is a two-state finite-state machine. Each state is associated with a state refinement labeled in the figure as a "time-based system." The state refinement defines dynamic behavior of the outputs and (possibly) additional continuous state variables. In addition, each transition can optionally specify set actions, which set the values of such additional state variables when a transition is taken. The example of Figure 4.3 is rather trivial, in that it has no continuous state variables, no output actions, and no set actions.

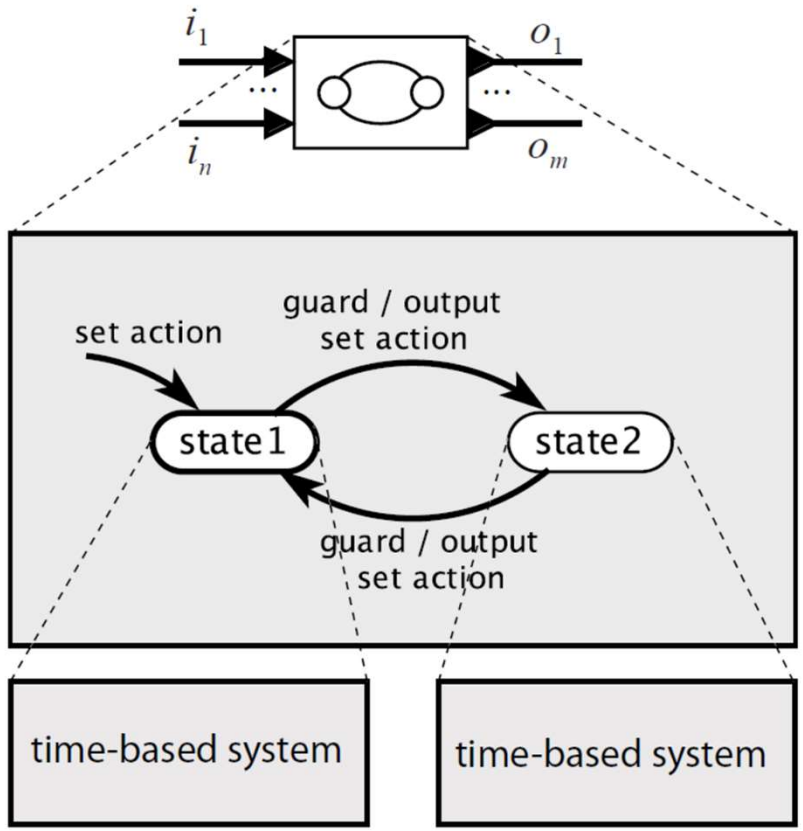

Figure 4.4: Notation for hybrid systems.

A hybrid system is sometimes called a **modal model** because it has a finite number of **mode**s, one for each state of the FSM, and when it is in a mode, it has dynamics specified by the state refinement. The states of the FSM may be referred to as modes rather than states, which as we will see, helps prevent confusion with states of the refinements.

The next simplest such dynamics, besides the rather trivial constant outputs of Example 4.2 is found in timed automata, which we discuss next.

4.2 Classes of Hybrid Systems

Hybrid systems can be quite elaborate. In this section, we first describe a relatively simple form known as timed automata. We then illustrate more elaborate forms that model nontrivial physical dynamics and nontrivial control systems.

4.2.1 Timed Automata

Most cyber-physical systems require measuring the passage of time and performing actions at specific times. A device that measures the passage of time, a **clock**, has a particularly simple dynamics: its state progresses linearly in time. In this section, we describe **timed automata**, a formalism introduced by Alur and Dill (1994), which enable the construction of more complicated systems from such simple clocks.

Timed automata are the simplest non-trivial hybrid systems. They are modal models where the time-based refinements have very simple dynamics; all they do is measure the passage of time. A clock is modeled by a first-order differential equation,

$$\forall\, t \in T_m, \quad \dot{s}(t) = a,$$

where $s\colon \mathbb{R} \to \mathbb{R}$ is a continuous-time signal, $s(t)$ is the value of the clock at time t, and $T_m \subset \mathbb{R}$ is the subset of time during which the hybrid system is in mode m. The rate of the clock, a, is a constant while the system is in this mode.[1]

[1] The variant of timed automata we describe in this chapter differs from the original model of Alur and Dill (1994) in that the rates of clocks in different modes can be different. This variant is sometimes described in the literature as *multi-rate* timed automata.

Example 4.3: Recall the thermostat of Example 4.1, which uses hysteresis to prevent chattering. An alternative implementation that would also prevent chattering would use a single temperature threshold, but instead would require that the heater remain on or off for at least a minimum amount of time, regardless of the temperature. This design would not have the hysteresis property, but may be useful nonetheless. This can be modeled as a timed automaton as shown in Figure 4.5. In that figure, each state refinement has a clock, which is a continuous-time signal s with dynamics given by

$$\dot{s}(t) = 1 \ .$$

The value $s(t)$ increases linearly with t. Note that in that figure, the state refinement is shown directly with the name of the state in the state bubble. This shorthand is convenient when the refinement is relatively simple.

Notice that the initial state cooling has a set action on the dangling transition indicating the initial state, written as

$$s(t) := T_c \ .$$

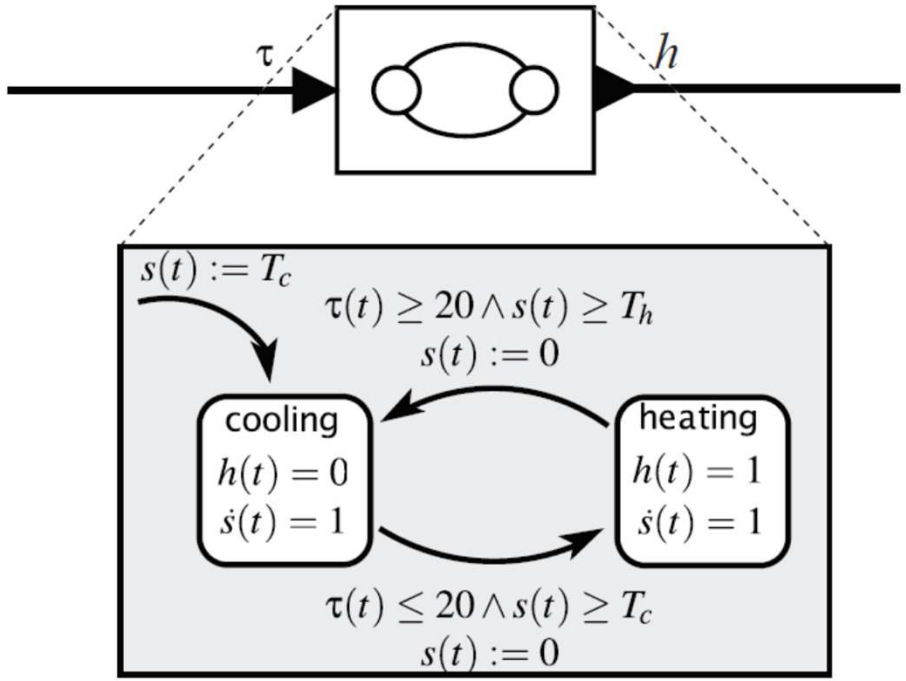

Figure 4.5: A timed automaton modeling a thermostat with a single temperature threshold and minimum times in each mode.

As we did with extended state machines, we use the notation ":=" to emphasize that this is an assignment, not a predicate. This action ensures that when the thermostat starts, it can immediately transition to the heating mode if the temperature $\tau(t)$ is less than or equal to 20 degrees. The other two transitions each have set actions that reset the clock s to zero. The portion of the guard that specifies $s(t) \geq T_h$ ensures that the heater will always be on for at least time T_h. The portion of the guard that specifies $s(t) \geq T_c$ specifies that once the heater goes off, it will remain off for at least time T_c.

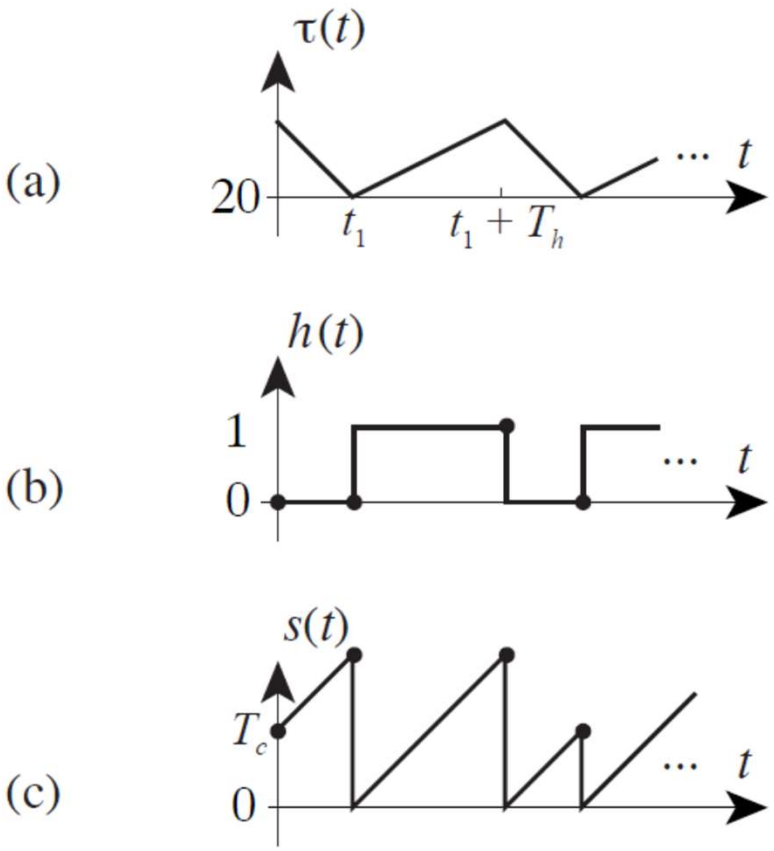

Figure 4.6: (a) A temperature input to the hybrid system of Figure 4.5, (b) the output h, and (c) the refinement state s.

A possible execution of this timed automaton is shown in Figure 4.6. In that figure, we assume that the temperature is initially above the setpoint of 20 degrees, so the FSM remains in the cooling state until the temperature drops to 20 degrees. At that time t_1, it can take the transition immediately because $s(t_1) > T_c$. The transition resets s to zero and turns on the heater. The heater will remain on until time $t_1 + T_h$, assuming that the temperature only rises when the heater is on. At time $t_1 + T_h$, it will transition back to the cooling state and turn the heater off. (We assume here that a transition is taken as soon as it is enabled. Other transition semantics are possible.) It will cool until at least time T_c elapses and until the temperature drops again to 20 degrees, at which point it will turn the heater back on.

In the previous example the state of system at any time t is not only the mode, heating or cooling, but also the current value $s(t)$ of the clock. We call s a **continuous state** variable. whereas heating and cooling are **discrete state**s. Thus. note that the term "state" for such a hybrid system can become confusing. The FSM has states, but so do the refinement systems (unless they are memoryless). When there is any possibility of confusion we explicitly refer to the states of the machine as modes.

Transitions between modes have actions associated with them. Sometimes, it is useful to have transitions from one mode back to itself, just so that the action can be realized. This is illustrated in the next example, which also shows a timed automaton that produces a pure output.

Example 4.4: The timed automaton in Figure 4.7 produces a pure output that will be present every T time units, starting at the time when the system begins executing. Notice that the guard on the transition, $s(t) \geq T$, is followed by an output action, *tick*, and a set action, $s(t) := 0$.

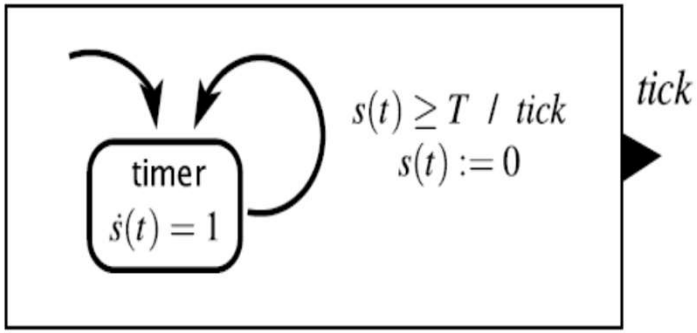

Figure 4.7: A timed automaton that generates a pure output event every T time units.

Figure 4.7 shows another notational shorthand that works well for simple diagrams. The automaton is shown directly inside the icon for its actor model.

Example 4.5: The traffic light controller of Figure 3.10 is a time triggered machine that assumes it reacts once each second. Figure 4.8 shows a timed automaton with the same behavior. It is more explicit about the passage of time in that its temporal dynamics do not depend on unstated assumptions about when the machine will react.

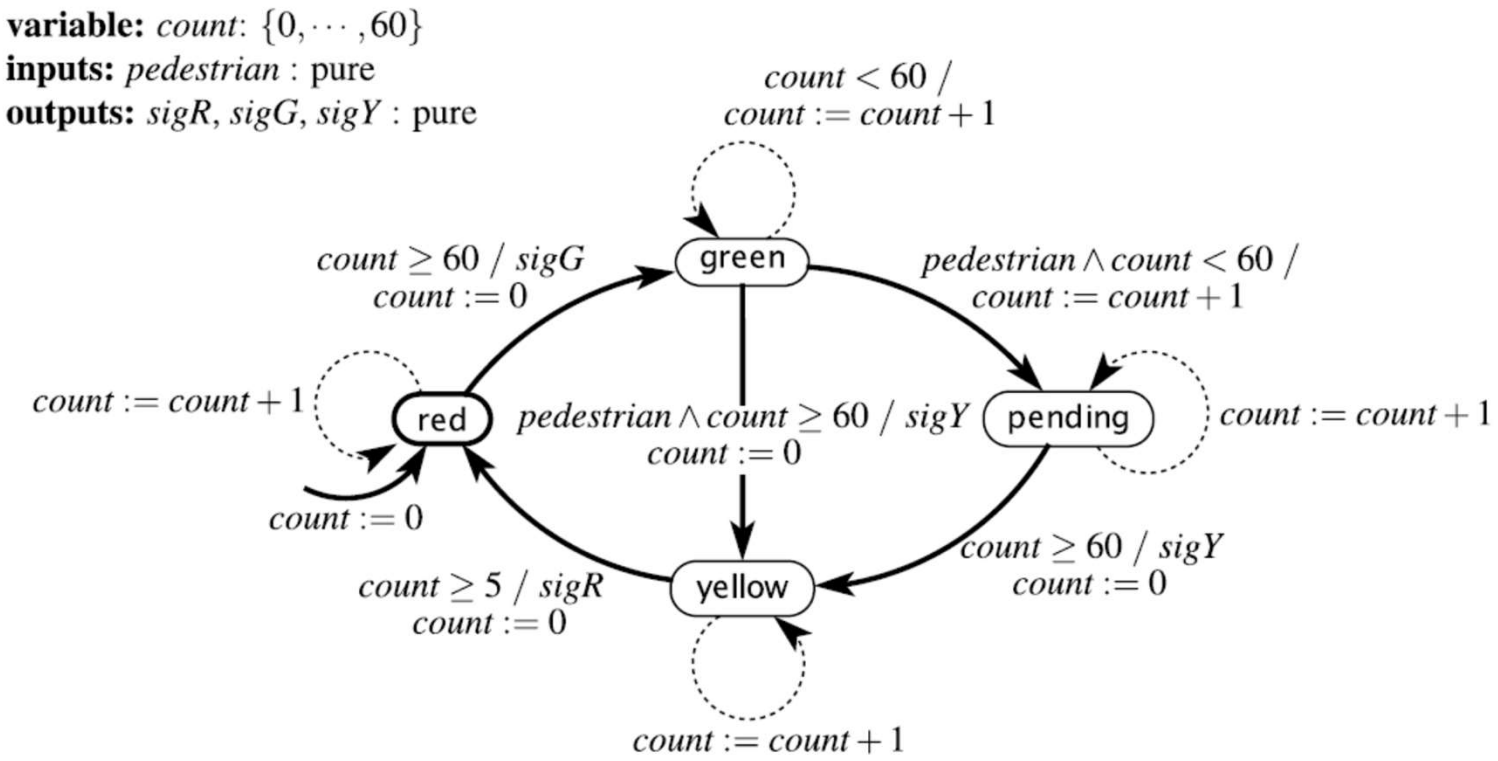

Figure 3.10: Extended state machine model of a traffic light controller that keeps track of the passage of time, assuming it reacts at regular intervals.

continuous variable: $x(t) : \mathbb{R}$
inputs: *pedestrian* : pure
outputs: *sigR*, *sigG*, *sigY* : pure

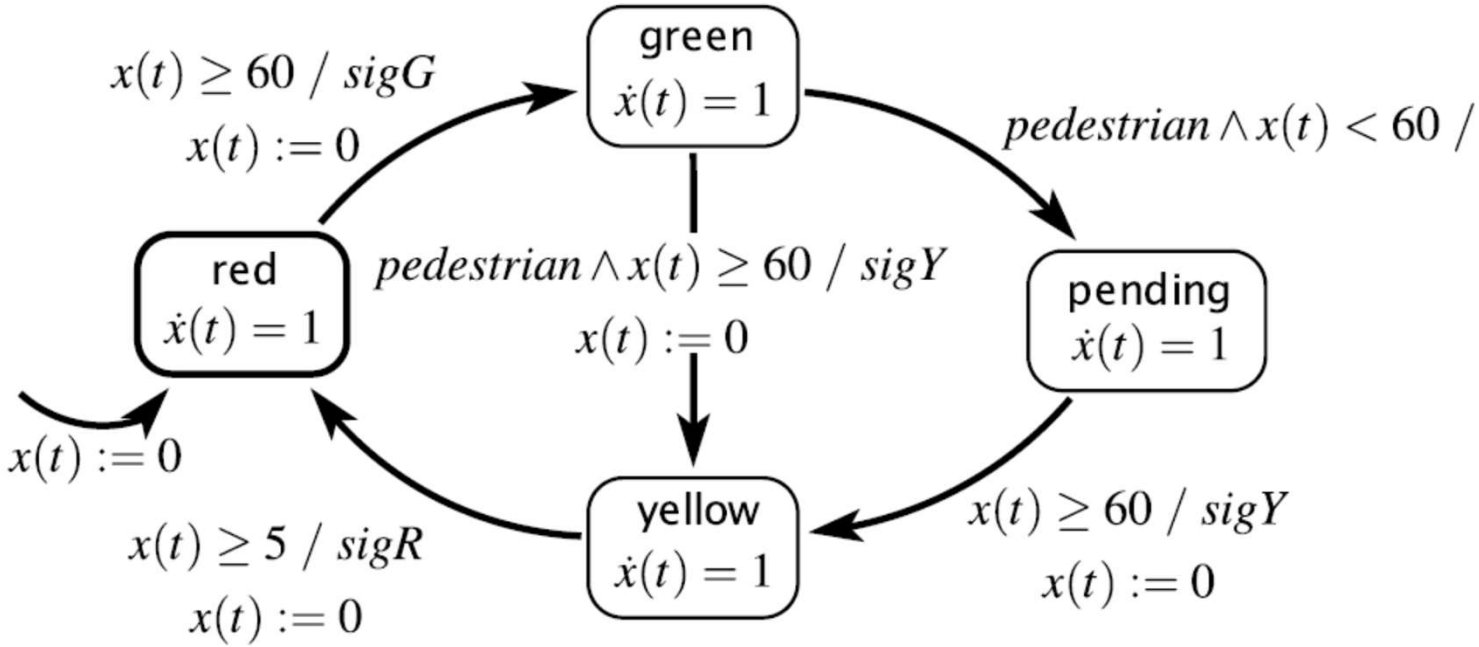

Figure 4.8: A timed automaton variant of the traffic light controller of Figure 3.10.

4.2.2 Higher-Order Dynamics

In timed automata, all that happens in the time-based refinement systems is that time passes. Hybrid systems, however, are much more interesting when the behavior of the refinements is more complex. Specifically,

As in Example 4.4, it is sometimes useful to have hybrid system models with only one state. The actions on one or more state transitions define the discrete event behavior that combines with the time-based behavior.

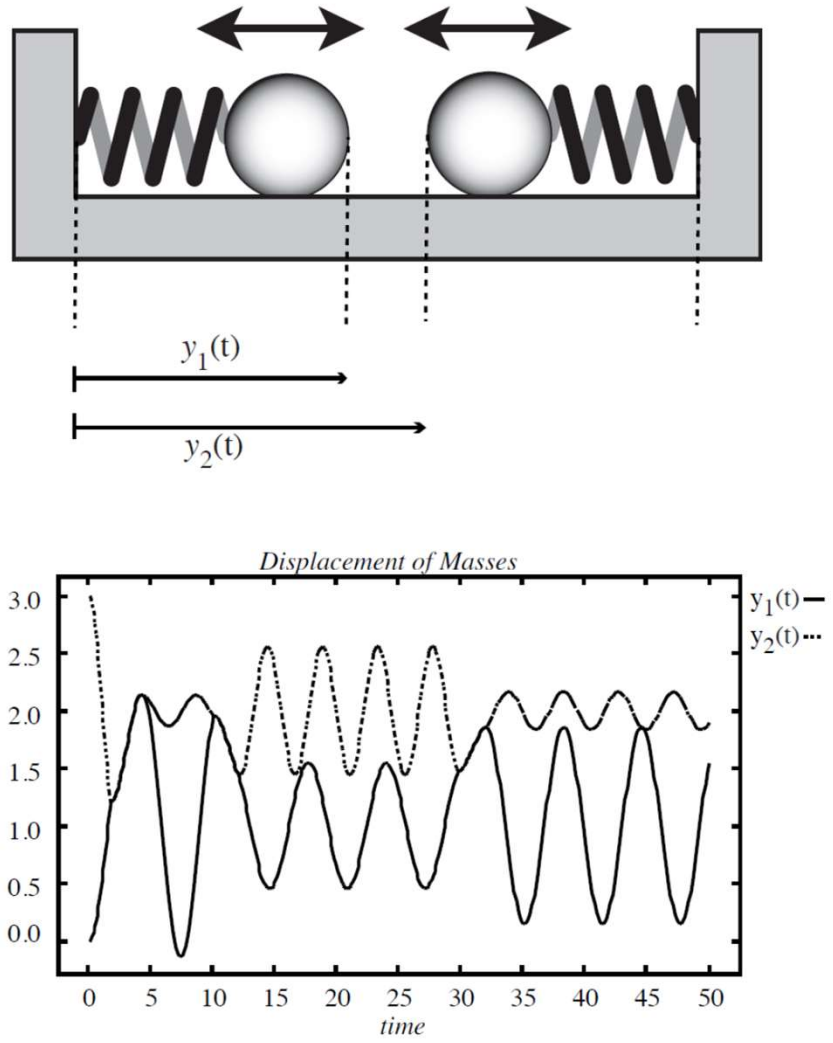

Figure 4.9: Sticky masses system considered in Example 4.6.

Example 4.6: Consider the physical system depicted in Figure 4.9. Two sticky round masses are attached to springs. The springs are compressed or extended and then released. The masses oscillate on a frictionless table. If they collide, they stick together and oscillate together. After some time, the stickiness decays, and masses pull apart again.

A plot of the displacement of the two masses as a function of time is shown in Figure 4.9. Both springs begin compressed, so the masses begin moving towards one another. They almost immediately collide, and then oscillate together for a brief period until they pull apart. In this plot, they collide two more times, and almost collide a third time.

The physics of this problem is quite simple if we assume idealized springs. Let $y_1(t)$ denote the right edge of the left mass at time t, and $y_2(t)$ denote the left edge of the right mass at time t, as shown in Figure 4.9. Let p_1 and p_2 denote the neutral positions of the two masses, i.e., when the springs are neither extended nor compressed, so the force is zero. For an ideal spring, the force at time t on the mass is proportional to $p_1 - y_1(t)$ (for the left mass) and $p_2 - y_2(t)$ (for the right mass). The force is positive to the right and negative to the left.

Let the spring constants be k_1 and k_2, respectively. Then the force on the left spring is $k_1(p_1 - y_1(t))$, and the force on the right spring is $k_2(p_2 - y_2(t))$. Let the masses be m_1 and m_2 respectively. Now we can use Newton's second law, which relates force, mass, and acceleration,

$$f = ma.$$

The acceleration is the second derivative of the position with respect to time, which we write $\ddot{y}_1(t)$ and $\ddot{y}_2(t)$. Thus, as long as the masses are separate, their dynamics are given by

$$\ddot{y}_1(t) = k_1(p_1 - y_1(t))/m_1 \tag{4.1}$$
$$\ddot{y}_2(t) = k_2(p_2 - y_2(t))/m_2. \tag{4.2}$$

When the masses collide, however, the situation changes. With the masses stuck together, they behave as a single object with mass $m_1 + m_2$. This single object is pulled in opposite directions by two springs. While the masses are stuck together, $y_1(t) = y_2(t)$. Let

$$y(t) = y_1(t) = y_2(t).$$

The dynamics are then given by

$$\ddot{y}(t) = \frac{k_1 p_1 + k_2 p_2 - (k_1 + k_2) y(t)}{m_1 + m_2}. \tag{4.3}$$

It is easy to see now how to construct a hybrid systems model for this physical system. The model is shown in Figure 4.10. It has two modes, apart and together. The refinement of the apart mode is given by (4.1) and (4.2), while the refinement of the together mode is given by (4.3).

We still have work to do, however, to label the transitions. The initial transition is shown in Figure 4.10 entering the apart mode. Thus, we are assuming the masses begin apart. Moreover, this transition is labeled with a set action that sets the initial positions of the two masses to i_1 and i_2 and the initial velocities to zero.

The transition from apart to together has the guard

$$y_1(t) = y_2(t) \ .$$

This transition has a set action which assigns values to two continuous state variables $y(t)$ and $\dot{y}(t)$, which will represent the motion of the two masses stuck together. The value it assigns to $\dot{y}(t)$ conserves momentum. The momentum of the left mass is $\dot{y}_1(t)m_1$, the momentum of the right mass is $\dot{y}_2(t)m_2$, and the momentum of the combined masses is $\dot{y}(t)(m_1+m_2)$. To make these equal, it sets

$$\dot{y}(t) = \frac{\dot{y}_1(t)m_1 + \dot{y}_2(t)m_2}{m_1+m_2}.$$

The refinement of the together mode gives the dynamics of y and simply sets $y_1(t) = y_2(t) = y(t)$, since the masses are moving together. The transition from apart to together sets $y(t)$ equal to $y_1(t)$ (it could equally well have chosen $y_2(t)$, since these are equal).

The transition from together to apart has the more complicated guard

$$(k_1 - k_2)y(t) + k_2 p_2 - k_1 p_1 > s,$$

where s represents the stickiness of the two masses. This guard is satisfied when the right-pulling force on the right mass exceeds the right-pulling force on the left mass by more than the stickiness. The right-pulling force on the right mass is simply

$$f_2(t) = k_2(p_2 - y(t))$$

and the right-pulling force on the left mass is

$$f_1(t) = k_1(p_1 - y(t)).$$

Thus,

$$f_2(t) - f_1(t) = (k_1 - k_2)y(t) + k_2 p_2 - k_1 p_1.$$

When this exceeds the stickiness s, then the masses pull apart.

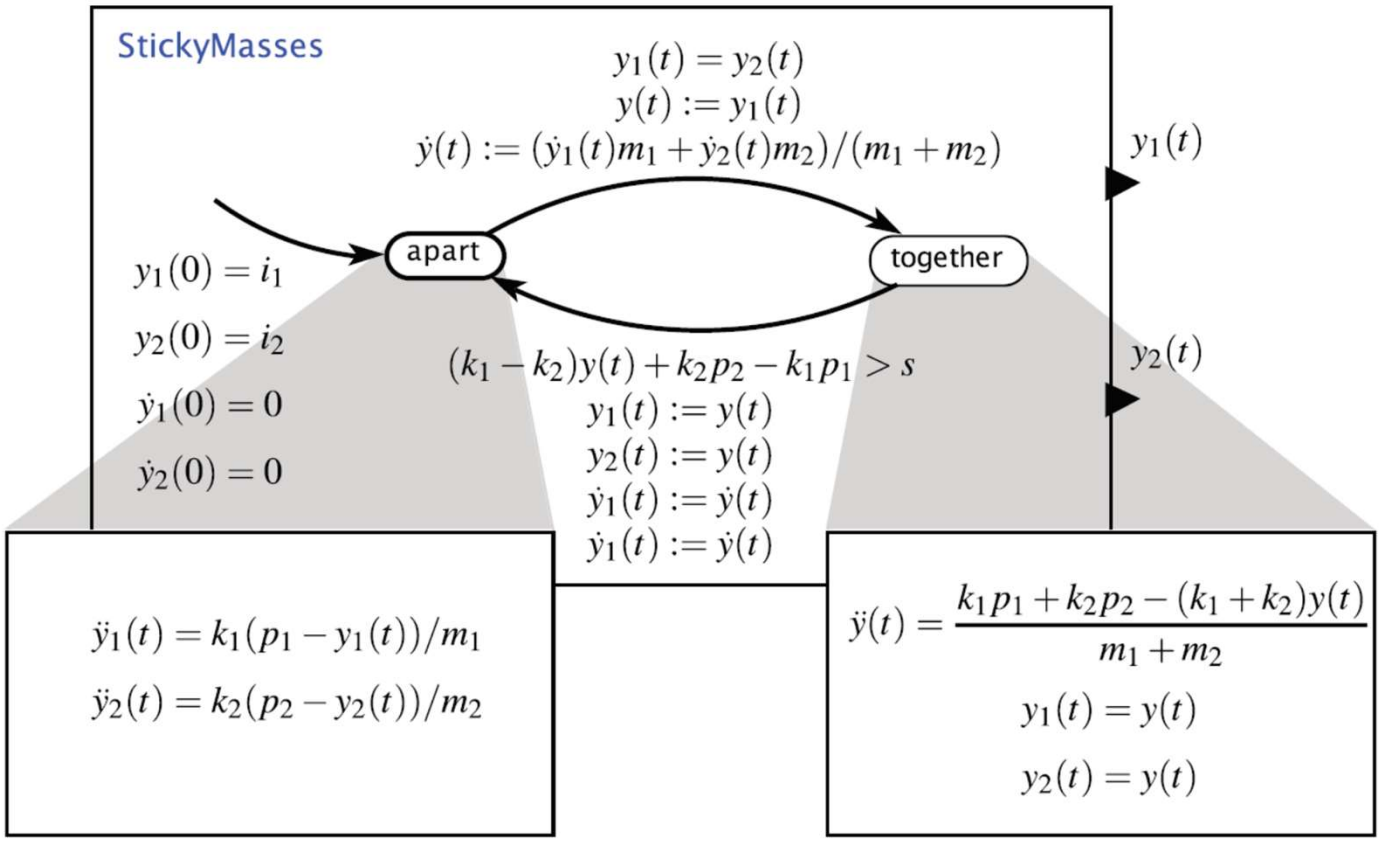

Figure 4.10: Hybrid system model for the sticky masses system considered in Example 4.6.

As in Example 4.4, it is sometimes useful to have hybrid system models with only one state. The actions on one or more state transitions define the discrete event behavior that combines with the time-based behavior.

Example 4.7: Consider a bouncing ball. At time $t = 0$, the ball is dropped from a height $y(0) = h_0$, where h_0 is the initial height in meters. It falls freely. At some later time t_1 it hits the ground with a velocity $\dot{y}(t_1) < 0$ m/s (meters per second). A *bump* event is produced when the ball hits the ground. The collision is **inelastic** (meaning that kinetic energy is lost), and the ball bounces back up with velocity $-a\dot{y}(t_1)$, where a is constant with $0 < a < 1$. The ball will then rise to a certain height and fall back to the ground repeatedly.

The behavior of the bouncing ball can be described by the hybrid system of Figure 4.11. There is only one mode, called free. When it is not in contact with the ground, we know that the ball follows the second-order differential equation,

$$\ddot{y}(t) = -g, \tag{4.4}$$

where $g = 9.81$ m/sec^2 is the acceleration imposed by gravity. The continuous state variables of the free mode are

$$s(t) = \begin{bmatrix} y(t) \\ \dot{y}(t) \end{bmatrix}$$

with the initial conditions $y(0) = h_0$ and $\dot{y}(0) = 0$. It is then a simple matter to rewrite (4.4) as a first-order differential equation,

$$\dot{s}(t) = f(s(t)) \tag{4.5}$$

for a suitably chosen function f.

At the time $t = t_1$ when the ball first hits the ground, the guard

$$y(t) = 0$$

is satisfied, and the self-loop transition is taken. The output *bump* is produced, and the set action $\dot{y}(t) := -a\dot{y}(t)$ changes $\dot{y}(t_1)$ to have value $-a\dot{y}(t_1)$. Then (4.4) is followed again until the guard becomes true again.

By integrating (4.4) we get, for all $t \in (0, t_1)$,

$$\begin{aligned} \dot{y}(t) &= -gt, \\ y(t) &= y(0) + \int_0^t \dot{y}(\tau)d\tau = h_0 - \frac{1}{2}gt^2. \end{aligned}$$

So $t_1 > 0$ is determined by $y(t_1) = 0$. It is the solution to the equation

$$h_0 - \frac{1}{2}gt^2 = 0.$$

Thus,

$$t_1 = \sqrt{2h_0/g}.$$

Figure 4.11 plots the continuous state versus time.

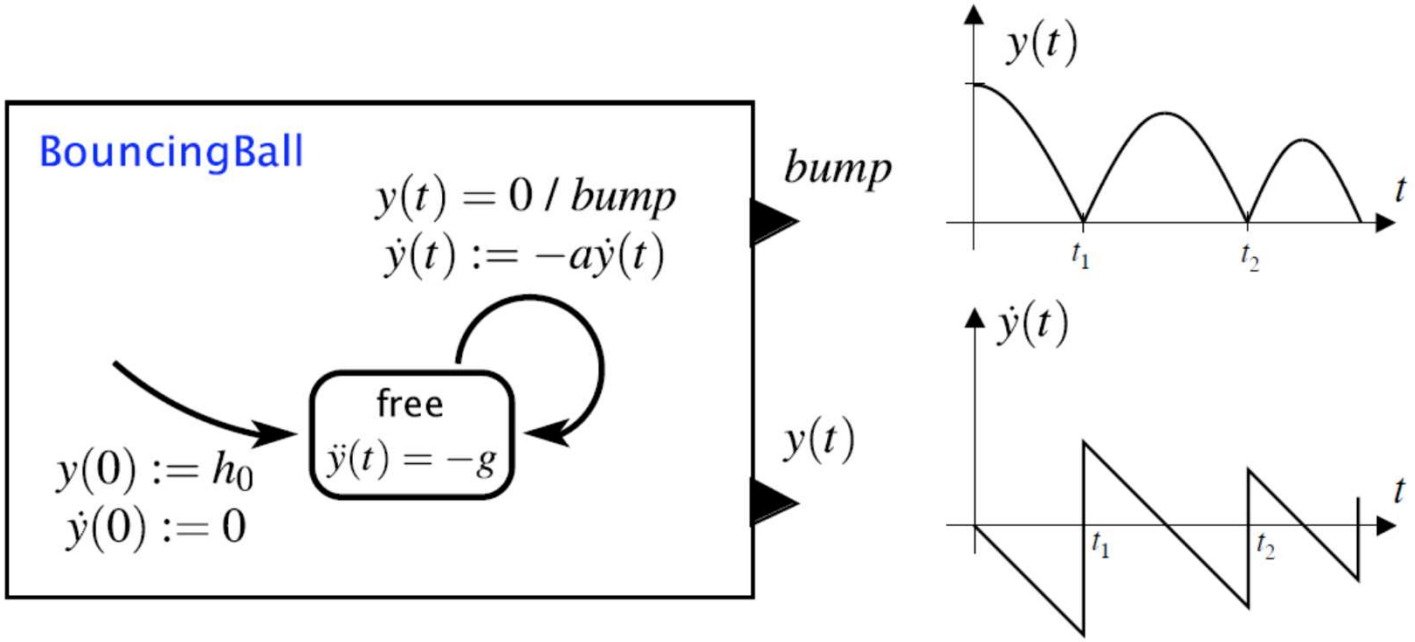

Figure 4.11: The motion of a bouncing ball may be described as a hybrid system with only one mode. The system outputs a *bump* each time the ball hits the ground, and also outputs the position of the ball. The position and velocity are plotted versus time at the right.

The bouncing ball example above has an interesting difficulty that is explored .

Specifically, the time between bounces gets smaller as time increases. In fact, it gets smaller fast enough that an infinite number of bounces occur in a finite amount of time. A system with an infinite number of discrete events in a finite amount of time is called a **Zeno** system, after Zeno of Elea, a pre-Socratic Greek philosopher famous for his paradoxes. In the physical world, of course, the ball will eventually stop bouncing. The Zeno behavior is an artifact of the model.

4.2.3 Supervisory control

A control system involves four components: a system called the **plant**, the physical process that is to be controlled; the environment in which the plant operates; the sensors that measure some variables of the plant and the environment; and the controller that determines the mode transition structure and selects the time-based inputs to the plant. The controller has two levels: the **supervisory control** that determines the mode transition structure, and the **low-level control** that determines the time-based inputs to the plant. Intuitively, the supervisory controller determines which of several strategies should be followed, and the low-level controller implements the selected strategy. Hybrid systems are ideal for modeling such two-level controllers. We show how through a detailed example.

Example 4.8: Consider an **automated guided vehicle** (**AGV**) that moves along a closed track painted on a warehouse or factory floor. We will design a controller so that the vehicle closely follows the track.

The vehicle has two degrees of freedom. At any time t, it can move forward along its body axis with speed $u(t)$ with the restriction that $0 \leq u(t) \leq 10$ mph (miles per hour). It can also rotate about its center of gravity with an angular speed $\omega(t)$ restricted to $-\pi \leq \omega(t) \leq \pi$ radians/second. We ignore the inertia of the vehicle, so we assume that we can instantaneously change the velocity or angular speed.

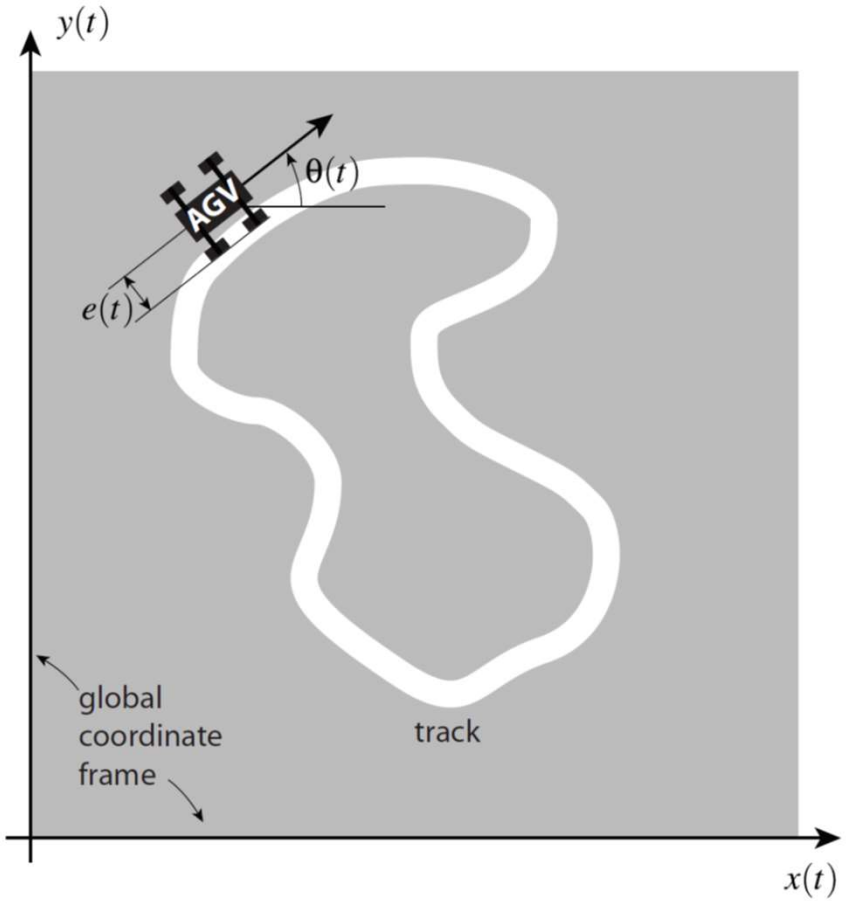

Figure 4.12: Illustration of the automated guided vehicle of Example 4.8. The vehicle is following a curved painted track, and has deviated from the track by a distance $e(t)$. The coordinates of the vehicle at time t with respect to the global coordinate frame are $(x(t), y(t), \theta(t))$.

Let $(x(t), y(t)) \in \mathbb{R}^2$ be the position relative to some fixed coordinate frame and $\theta(t) \in (-\pi, \pi]$ be the angle (in radians) of the vehicle at time t, as shown in Figure 4.12. In terms of this coordinate frame, the motion of the vehicle is given by a system of three differential equations,

$$\begin{aligned} \dot{x}(t) &= u(t)\cos\theta(t), \\ \dot{y}(t) &= u(t)\sin\theta(t), \\ \dot{\theta}(t) &= \omega(t). \end{aligned} \tag{4.6}$$

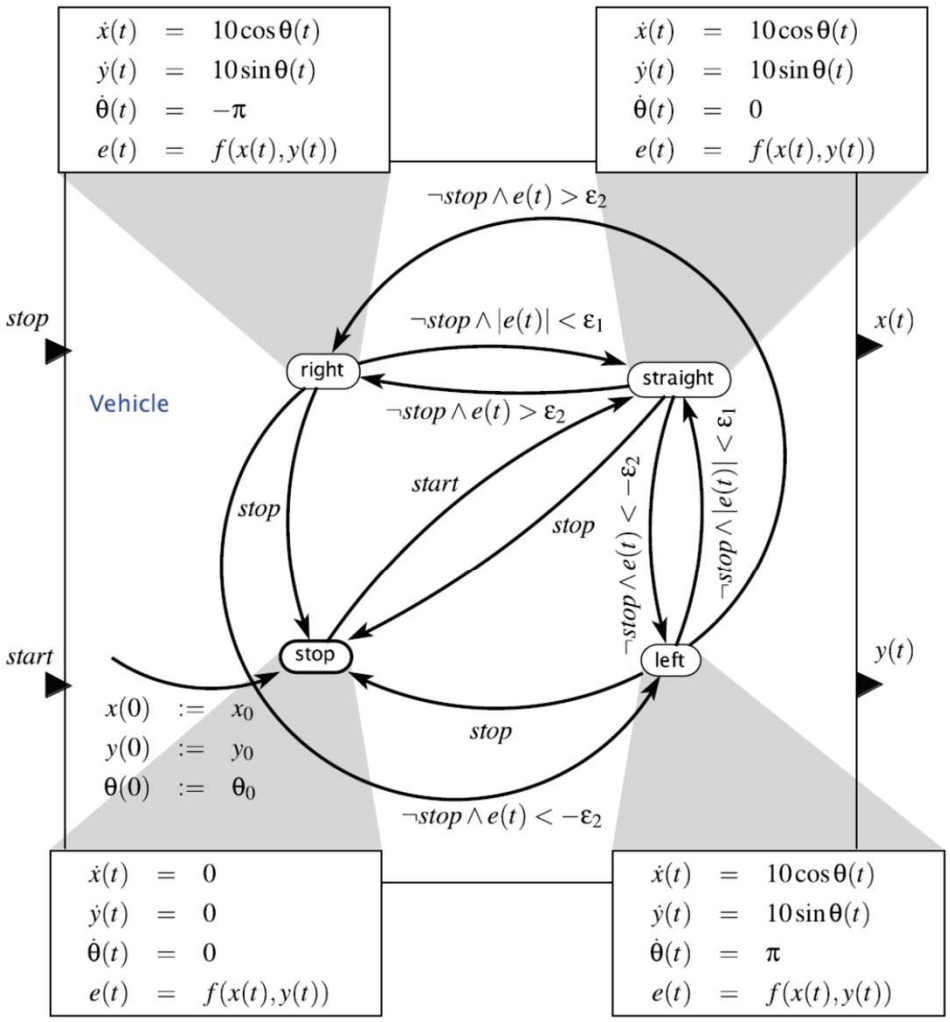

Figure 4.13: The automatic guided vehicle of Example 4.8 has four modes: stop, straight, left, right.

Equations (4.6) describe the plant. The environment is the closed painted track. It could be described by an equation. We will describe it indirectly below by means of a sensor.

The two-level controller design is based on a simple idea. The vehicle always moves at its maximum speed of 10 mph. If the vehicle strays too far to the left of the track, the controller steers it towards the right; if it strays too far to the right of the track, the controller steers it towards the left. If the vehicle is close to the track, the controller maintains the vehicle in a straight direction. Thus the controller guides the vehicle in four modes, left, right, straight, and stop. In stop mode, the vehicle comes to a halt.

The following differential equations govern the AGV's motion in the refinements of the four modes. They describe the low-level controller, i.e., the selection of the time-based plant inputs in each mode.

straight

$$\begin{aligned}\dot{x}(t) &= 10\cos\theta(t)\\ \dot{y}(t) &= 10\sin\theta(t)\\ \dot{\theta}(t) &= 0\end{aligned}$$

left

$$\begin{aligned}\dot{x}(t) &= 10\cos\theta(t)\\ \dot{y}(t) &= 10\sin\theta(t)\\ \dot{\theta}(t) &= \pi\end{aligned}$$

right

$$\begin{aligned}\dot{x}(t) &= 10\cos\theta(t)\\ \dot{y}(t) &= 10\sin\theta(t)\\ \dot{\theta}(t) &= -\pi\end{aligned}$$

stop

$$\begin{aligned}\dot{x}(t) &= 0\\ \dot{y}(t) &= 0\\ \dot{\theta}(t) &= 0\end{aligned}$$

In the stop mode, the vehicle is stopped, so $x(t)$, $y(t)$, and $\theta(t)$ are constant. In the left mode, $\theta(t)$ increases at the rate of π radians/second, so from Figure 4.12 we see that the vehicle moves to the left. In the right mode, it moves to the right. In the straight mode, $\theta(t)$ is constant, and the vehicle moves straight ahead with a constant heading. The refinements of the four modes are shown in the boxes of Figure 4.13.

We design the supervisory control governing transitions between modes in such a way that the vehicle closely follows the track, using a sensor that determines how far the vehicle is to the left or right of the track. We can build such a sensor using photodiodes. Let's suppose the track is painted with a light-reflecting color, whereas the floor is relatively dark. Underneath the AGV we place an array of photodiodes as shown in Figure 4.14. The array is perpendicular to the AGV body axis. As the AGV passes over the track, the diode directly above the track generates more current than the other diodes. By comparing the magnitudes of the currents through the different diodes, the sensor estimates the displacement $e(t)$ of the center of the array (hence, the center of the AGV) from the track. We adopt the convention that $e(t) < 0$ means that the AGV is to the right of the track and $e(t) > 0$ means it is to the left. We model the sensor output as a function f of the AGV's position,

$$\forall t, \quad e(t) = f(x(t), y(t)).$$

The function f of course depends on the environment—the track. We now specify the supervisory controller precisely. We select two thresholds, $0 < \varepsilon_1 < \varepsilon_2$, as shown in Figure 4.14. If the magnitude of the displacement is small, $|e(t)| < \varepsilon_1$, we consider that the AGV is close enough to the track, and the AGV can move straight ahead, in straight mode. If $e(t) > \varepsilon_2$ ($e(t)$ is large and positive), the AGV has strayed too far to the left and must be steered to the right, by switching to right mode. If $e(t) < -\varepsilon_2$ ($e(t)$ is large and negative), the AGV has strayed too far to the right and must be steered to the left, by switching to left mode. This control logic is captured in the mode transitions of Figure 4.13. The inputs are pure signals *stop* and *start*. These model an operator that can stop or start the AGV. There is no continuous-time input. The outputs represent the position of the vehicle, $x(t)$ and $y(t)$. The initial mode is stop, and the initial values of its refinement are (x_0, y_0, θ_0).

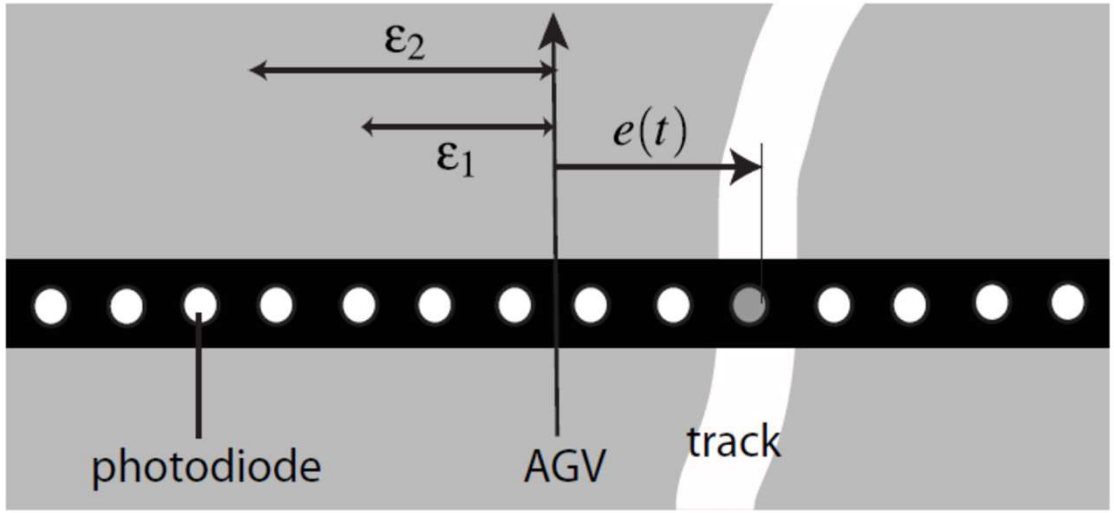

Figure 4.14: An array of photodiodes under the AGV is used to estimate the displacement e of the AGV relative to the track. The photodiode directly above the track generates more current.

We analyze how the AGV will move. Figure 4.15 sketches one possible trajectory. Initially the vehicle is within distance ε_1 of the track, so it moves straight. At some later time, the vehicle goes too far to the left, so the guard

$$\neg stop \wedge e(t) > \varepsilon_2$$

is satisfied, and there is a mode switch to right. After some time, the vehicle will again be close enough to the track, so the guard

$$\neg stop \wedge |e(t)| < \varepsilon_1$$

is satisfied, and there is a mode switch to straight. Some time later, the vehicle is too far to the right, so the guard

$$\neg stop \wedge e(t) < -\varepsilon_2$$

is satisfied, and there is a mode switch to left. And so on.

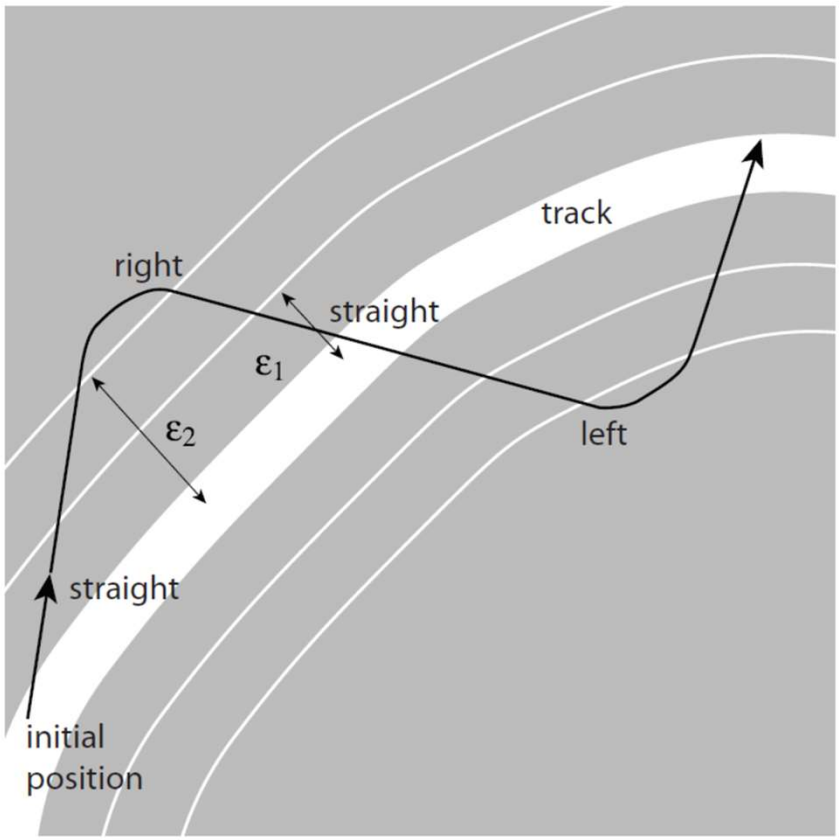

Figure 4.15: A trajectory of the AGV, annotated with modes.

The example illustrates the four components of a control system. The plant is described by the differential equations (4.6) that govern the evolution of the continuous state at time t, $(x(t), y(t), \theta(t))$, in terms of the plant inputs u and ω. The second component is the environment—the closed track. The third component is the sensor, whose output at time t, $e(t) = f(x(t), y(t))$, gives the position of the AGV relative to the track. The fourth component is the two-level controller. The supervisory controller comprises the four modes and the guards that determine when to switch between modes. The low-level controller specifies how the time-based inputs to the plant, u and ω, are selected in each mode.

4.3 Summary

Hybrid systems provide a bridge between time-based models and state-machine models. The combination of the two families of models provides a rich framework for describing real-world systems. There are two key ideas. First, discrete events are embedded in a time base. Second, a hierarchical description is particularly useful, where the system undergoes discrete transitions between different modes of operation. Associated with each mode of operation is a time-based system called the refinement of the mode. Mode transitions are taken when guards that specify the combination of inputs and continuous states are satisfied. The action associated with a transition, in turn, sets the continuous state in the destination mode.

The behavior of a hybrid system is understood using the tools of state machine analysis for mode transitions and the tools of time-based analysis for the refinement systems. The design of hybrid systems similarly proceeds on two levels: state machines are designed to achieve the appropriate logic of mode transitions, and refinement systems are designed to secure the desired time-based behavior in each mode.

Composition of State Machines

State machines provide a convenient way to model behaviors of systems. One disadvantage that they have is that for most interesting systems, the number of states is very large, often even infinite. Automated tools can handle large state spaces, but humans have more difficulty with any direct representation of a large state space.

A time-honored principle in engineering is that complicated systems should be described as compositions of simpler systems. This chapter gives a number of ways to do this with state machines. The reader should be aware, however, that there are many subtly different ways to compose state machines. Compositions that look similar on the surface may mean different things to different people. The rules of notation of a model are called its **syntax**, and the meaning of the notation is called its **semantics**. We will see that the same syntax can have many different semantics, which can cause no end of confusion.

Example 5.1: A now popular notation for concurrent composition of state machines called Statecharts was introduced by Harel (1987). Although they are all based on the same original paper, many variants of Statecharts have evolved (von der Beeck, 1994). These variants often assign different semantics to the same syntax.

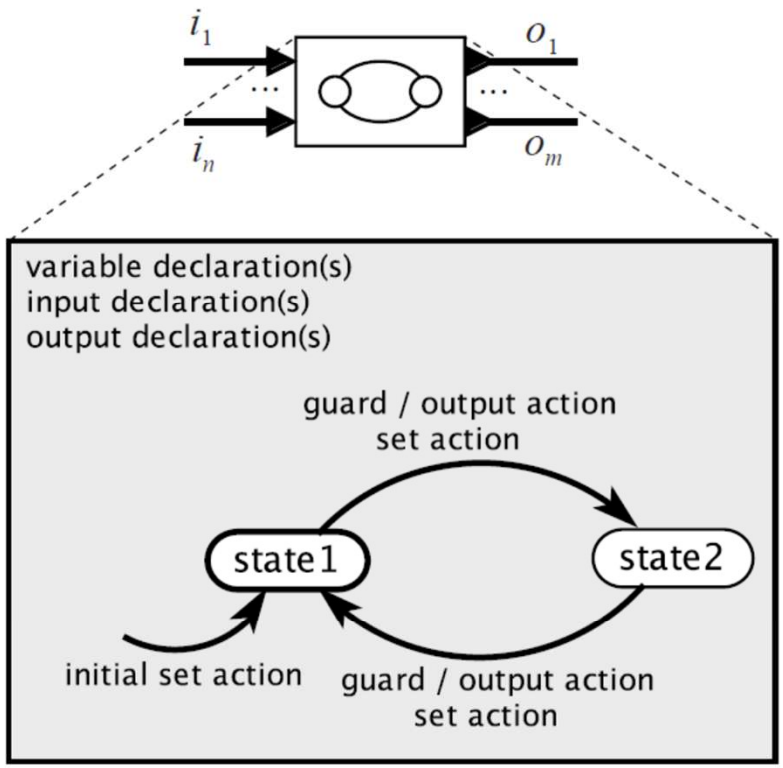

Figure 5.1: Summary of notation for state machines used in this chapter.

In this chapter, we assume an actor model for extended state machines using the syntax summarized in Figure 5.1. The semantics of a single such state machine is described in Chapter 3. This chapter will discuss the semantics that can be assigned to compositions of multiple such machines.

The first composition technique we consider is concurrent composition. Two or more state machines react either simultaneously or independently. If the reactions are simultaneous, we call it **synchronous composition**. If they are independent, then we call it **asynchronous composition**. But even within these classes of composition, many subtle variations in the semantics are possible. These variations mostly revolve around whether and how the state machines communicate and share variables.

The second composition technique we will consider is hierarchy. Hierarchical state machines can also enable complicated systems to be described as compositions of simpler systems. Again, we will see that subtle differences in semantics are possible.

5.1 Concurrent Composition

To study concurrent composition of state machines, we will proceed through a sequence of patterns of composition. These patterns can be combined to build arbitrarily complicated systems. We begin with the simplest case, side-by-side composition, where the state machines being composed do not communicate. We then consider allowing communication through shared variables, showing that this creates significant subtleties that can complicate modeling. We then consider communication through ports, first looking at serial composition, then expanding to arbitrary interconnections. We consider both synchronous and asynchronous composition for each type of composition.

5.1.1 Side-by-Side Synchronous Composition

The first pattern of composition that we consider is **side-by-side composition**, illustrated for two actors in Figure 5.2. In this pattern, we assume that the inputs and outputs of the two actors are disjoint, i.e., that the state machines do not communicate. In the figure, actor A has input i_1 and output o_1, and actor B has input i_2 and output o_2. The composition of the two actors is itself an actor C with inputs i_1 and i_2 and outputs o_1 and o_2.[1]

[1] The composition actor C may rename these input and output ports, but here we assume it uses the same names as the component actors.

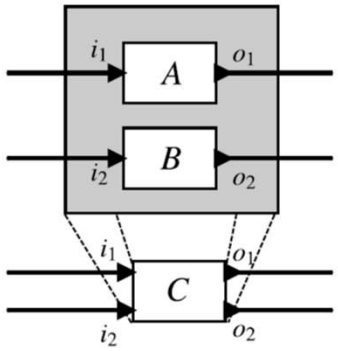

Figure 5.2: Side-by-side composition of two actors.

In the simplest scenario, if the two actors are extended state machines with variables, then those variables are also disjoint. We will later consider what happens when the two state machines share variables. Under **synchronous composition**, a reaction of C is a simultaneous reaction of A and B.

Example 5.2: Consider FSMs A and B in Figure 5.3. A has a single pure output a, and B has a single pure output b. The side-by-side composition C has two pure outputs, a and b. If the composition is synchronous, then on the first reaction, a will be *absent* and b will be *present*. On the second reaction, it will be the reverse. On subsequent reactions, a and b will continue to alternate being present.

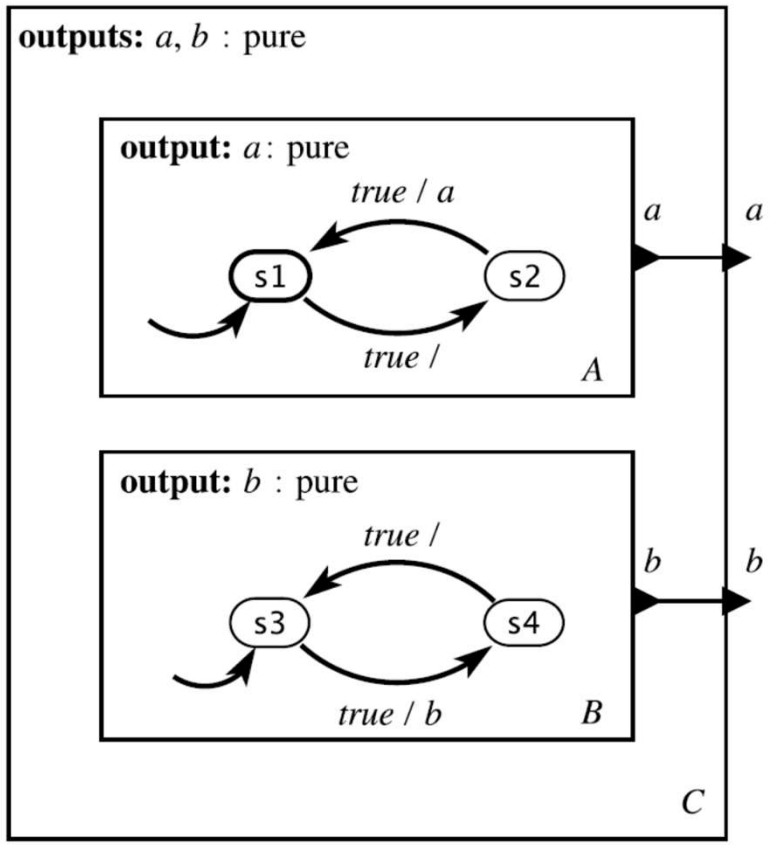

Figure 5.3: Example of side-by-side composition of two actors.

Synchronous side-by-side composition is simple for several reasons. First, recall from Section 3.3.2 that the environment determines when a state machine reacts. In synchronous side-by-side composition, the environment need not be aware that C is a composition of two state machines. Such compositions are **modular** in the sense that the composition itself becomes a component that can be further composed as if it were itself an atomic component.

Moreover, if the two state machines A and B are determinate, then the synchronous side-by-side composition is also determinate. We say that a property is **compositional** if a property held by the components is also a property of the composition. For synchronous side-by-side composition, determinacy is a compositional property.

In addition, a synchronous side-by-side composition of finite state machines is itself an FSM. A rigorous way to give the semantics of the composition is to define a single state machine for the composition. Suppose that as in Section 3.3.3, state machines A and B are given by the five tuples,

$$\begin{aligned} A &= (States_A, Inputs_A, Outputs_A, update_A, initialState_A) \\ B &= (States_B, Inputs_B, Outputs_B, update_B, initialState_B) \ . \end{aligned}$$

Then the synchronous side-by-side composition C is given by

$$States_C = States_A \times States_B \tag{5.1}$$

$$Inputs_C = Inputs_A \times Inputs_B \tag{5.2}$$

$$Outputs_C = Outputs_A \times Outputs_B \tag{5.3}$$

$$initialState_C = (initialState_A, initialState_B) \tag{5.4}$$

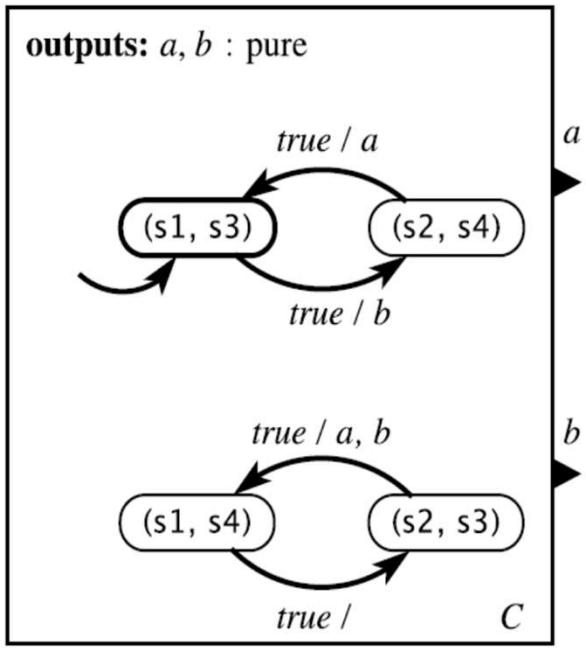

Figure 5.4: Single state machine giving the semantics of synchronous side-by-side composition of the state machines in Figure 5.3.

and the update function is defined by

$$update_C((s_A, s_B), (i_A, i_B)) = ((s'_A, s'_B), (o_A, o_B)),$$

where

$$(s'_A, o_A) = update_A(s_A, i_A),$$

and

$$(s'_B, o_B) = update_B(s_B, i_B),$$

for all $s_A \in States_A$, $s_B \in States_B$, $i_A \in Inputs_A$, and $i_B \in Inputs_B$.

Recall that $Inputs_A$ and $Inputs_B$ are sets of valuations. Each valuation in the set is an assignment of values to ports. What we mean by

$$Inputs_C = Inputs_A \times Inputs_B$$

is that a valuation of the inputs of C must include *both* valuations for the inputs of A and the inputs of B.

As usual, the single FSM C can be given pictorially rather than symbolically, as illustrated in the next example.

Example 5.3: The synchronous side-by-side composition C in Figure 5.3 is given as a single FSM in Figure 5.4. Notice that this machine behaves exactly as described in Example 5.2. The outputs a and b alternate being present. Notice further that $(\mathsf{s1}, \mathsf{s4})$ and $(\mathsf{s2}, \mathsf{s3})$ are not reachable states.

Side-by-Side Asynchronous Composition

In an **asynchronous composition** of state machines, the component machines react independently. This statement is rather vague, and in fact, it has several different interpretations. Each interpretation gives a semantics to the composition. The key to each semantics is how to define a reaction of the composition C in Figure 5.2. Two possibilities are:

- **Semantics 1.** A reaction of C is a reaction of one of A or B, where the choice is nondeterministic.
- **Semantics 2.** A reaction of C is a reaction of A, B, or both A and B, where the choice is nondeterministic. A variant of this possibility might allow *neither* to react.

Semantics 1 is referred to as an **interleaving semantics**, meaning that A or B never react simultaneously. Their reactions are interleaved in some order.

A significant subtlety is that under these semantics machines A and B may completely miss input events. That is, an input to C destined for machine A may be present in a reaction where the nondeterministic choice results in B reacting rather than A. If this is not desirable, then some control over scheduling or synchronous composition becomes a better choice.

Example 5.4: For the example in Figure 5.3, semantics 1 results in the composition state machine shown in Figure 5.5. This machine is nondeterministic. From state (s1, s3), when C reacts, it can move to (s2, s3) and emit no output, or it can move to (s1, s4) and emit b. Note that if we had chosen semantics 2, then it would also be able to move to (s2, s4).

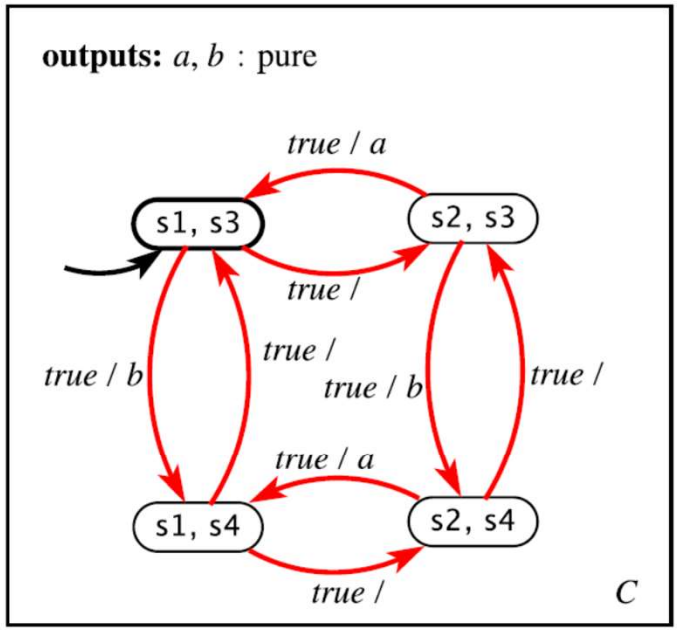

Figure 5.5: State machine giving the semantics of asynchronous side-by-side composition of the state machines in Figure 5.3.

For asynchronous composition under semantics 1, the symbolic definition of C has the same definitions of $States_C$, $Inputs_C$, $Outputs_C$, and $initialState_C$ as for synchronous composition, given in (5.1) through (5.4). But the update function differs, becoming

$$update_C((s_A, s_B), (i_A, i_B)) = ((s'_A, s'_B), (o'_A, o'_B)),$$

where either

$$(s'_A, o'_A) = update_A(s_A, i_A) \text{ and } s'_B = s_B \text{ and } o'_B = absent$$

or

$$(s'_B, o'_B) = update_B(s_B, i_B) \text{ and } s'_A = s_A \text{ and } o'_A = absent$$

for all $s_A \in States_A$, $s_B \in States_B$, $i_A \in Inputs_A$, and $i_B \in Inputs_B$. What we mean by $o'_B = absent$ is that all outputs of B are absent. Semantics 2 can be similarly defined

5.1.3 Shared Variables

An extended state machine has local variables that can be read and written as part of taking a transition. Sometimes it is useful when composing state machines to allow these variables to be shared among a group of machines. In particular, such shared variables can be useful for modeling interrupts, studied in Chapter 9, and threads, studied in Chapter 10. However, considerable care is required to ensure that the semantics of the model conforms with that of the program containing interrupts or threads. Many complications arise, including the memory consistency model and the notion of atomic operations.

Scheduling Semantics for Asynchronous Composition

In the case of semantics 1 and 2 given in Section 5.1.2, the choice of which component machine reacts is nondeterministic. The model does not express any particular constraints. It is often more useful to introduce some scheduling policies, where the environment is able to influence or control the nondeterministic choice. This leads to two additional possible semantics for asynchronous composition:

- **Semantics 3.** A reaction of C is a reaction of one of A or B, where the environment chooses which of A or B reacts.
- **Semantics 4.** A reaction of C is a reaction of A, B, or both A and B, where the choice is made by the environment.

Like semantics 1, semantics 3 is an interleaving semantics.

In one sense, semantics 1 and 2 are more compositional than semantics 3 and 4. To implement semantics 3 and 4, a composition has to provide some mechanism for the environment to choose which component machine should react (for scheduling the component machines). This means that the hierarchy suggested in Figure 5.2 does not quite work. Actor C has to expose more of its internal structure than just the ports and the ability to react.

In another sense, semantics 1 and 2 are less compositional than semantics 3 and 4 because determinacy is not preserved by composition. A composition of determinate state machines is not a determinate state machine.

Notice further that semantics 1 is an abstraction of semantics 3 in the sense that every behavior under semantics 3 is also a behavior under semantics 1. This notion of abstraction is studied in detail in Chapter 13.

The subtle differences between these choices make asynchronous composition rather treacherous. Considerable care is required to ensure that it is clear which semantics is used.

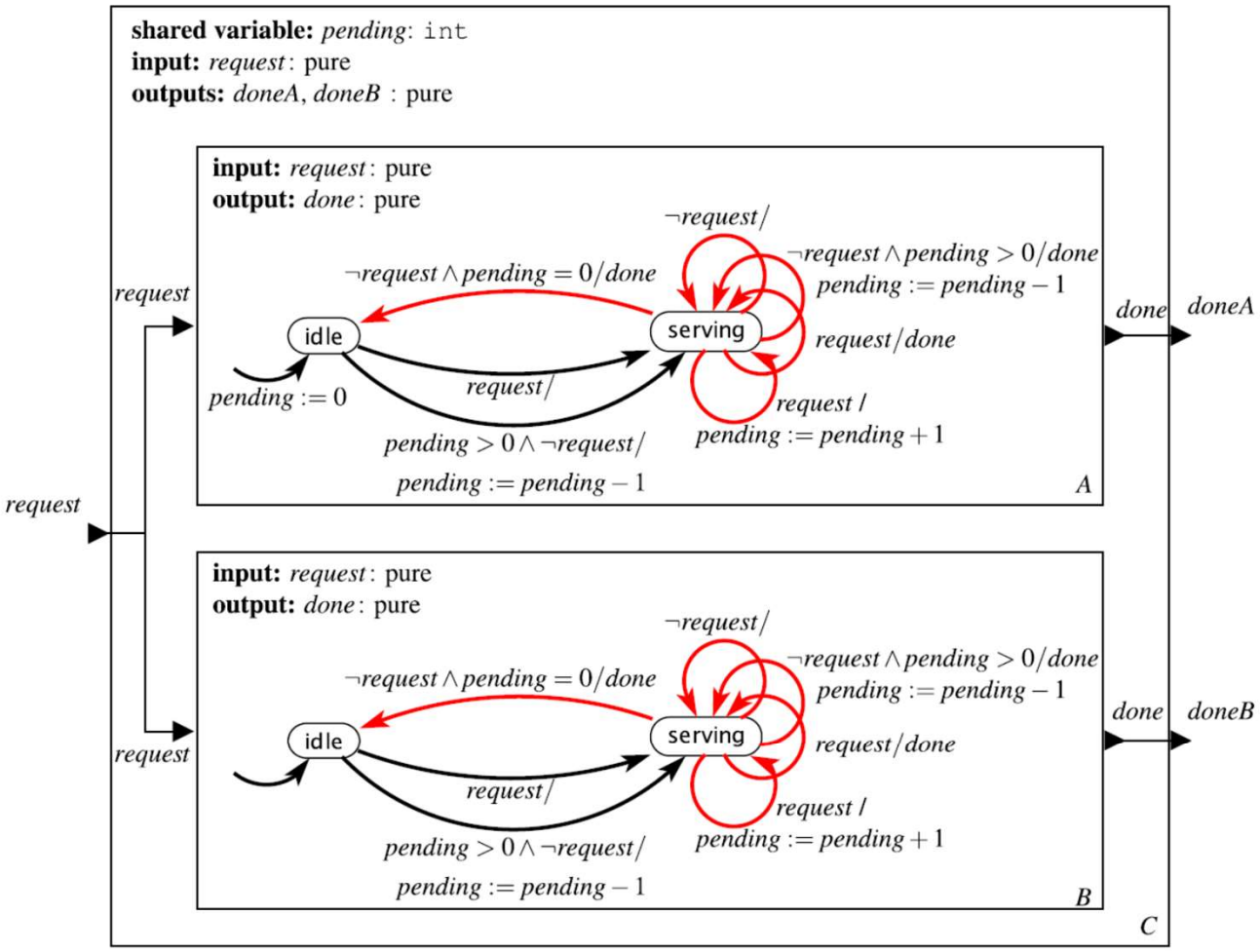

Figure 5.6: Model of two servers with a shared task queue, assuming asynchronous composition under semantics 1.

Example 5.5: Consider two servers that can receive requests from a network. Each request requires an unknown amount of time to service, so the servers share a queue of requests. If one server is busy, the other server can respond to a request, even if the request arrives at the network interface of the first server. This scenario fits a pattern similar to that in Figure 5.2, where A and B are the servers. We can model the servers as state machines as shown in Figure 5.6. In this model, a shared variable *pending* counts the number of pending job requests. When a request arrives at the composite machine C, one of the two servers is nondeterministically chosen to react, assuming asynchronous composition under semantics 1. If that server is idle, then it proceeds to serve the request. If the server is serving another request, then one of two things can happen: it can coincidentally finish serving the request it is currently serving, issuing the output *done*, and proceed to serve the new one, or it can increment the count of pending requests and continue to serve the current request. The choice between these is nondeterministic, to model the fact that the time it takes to service a request is unknown.

If C reacts when there is no request, then again either server A or B will be selected nondeterministically to react. If the server that reacts is idle and there are one or more pending requests, then the server transitions to serving and decrements the variable *pending*. If the server that reacts is not idle, then one of three things can happen. It may continue serving the current request, in which case it simply transitions on the self transition back to serving. Or it may finish serving the request, in which case it will transition to idle if there are no pending requests, or transition back to serving and decrement *pending* if there are pending requests.

The model in the previous example exhibits many subtleties of concurrent systems. First, because of the interleaving semantics, accesses to the shared variable are atomic operations, something that is quite challenging to guarantee in practice, as discussed in Chapters 9 and 10. Second, the choice of semantics 1 is reasonable in this case because the input goes to both of the component machines, so regardless of which component machine reacts, no input event will be missed. However, this semantics would not work if the two machines had independent inputs, because then requests could be missed. Semantics 2 can help prevent that, but what strategy should be used by the environment to determine which machine reacts? What if the two independent inputs both have requests present at the same reaction of C? If we choose semantics 4 in the sidebar on page 117 to allow both machines to react simultaneously, then what is the meaning when both machines update the shared variable? The updates are no longer atomic, as they are with an interleaving semantics.

Note further that choosing asynchronous composition under semantics 1 allows behaviors that do not make good use of idle machines. In particular, suppose that machine A is serving, machine B is idle, and a *request* arrives. If the nondeterministic choice results in machine A reacting, then it will simply increment *pending*. Not until the nondeterministic choice results in B reacting will the idle machine be put to use. In fact, semantics 1 allows behaviors that never use one of the machines.

Shared variables may be used in synchronous compositions as well, but sophisticated subtleties again emerge. In particular, what should happen if in the same reaction one machine reads a shared variable to evaluate a guard and another machine writes to the shared variable? Do we require the write before the read? What if the transition doing the write to the shared variable also reads the same variable in its guard expression? One possibility is to choose a **synchronous interleaving semantics**, where the component machines react in arbitrary order, chosen nondeterministically. This strategy has the disadvantage that a composition of two deterministic machines may be nondeterministic. An alternative version of the synchronous interleaving semantics has the component machines react in a fixed order determined by the environment or by some additional mechanism such as priority.

The difficulties of shared variables, particularly with asynchronous composition, reflect the inherent complexity of concurrency models with shared variables. Clean solutions require a more sophisticated semantics, to be discussed in Chapter 6. Specifically, in that chapter, we will explain the synchronous-reactive model of computation, which gives a synchronous composition semantics that is reasonably compositional.

So far, we have considered composition of machines that do not directly communicate. We next consider what happens when the outputs of one machine are the inputs of another.

5.1.4 Cascade Composition

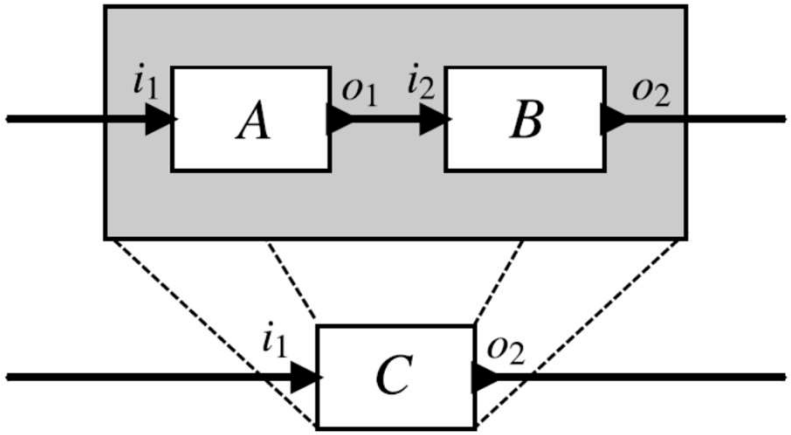

Figure 5.7: Cascade composition of two actors.

Consider two state machines A and B that are composed as shown in Figure 5.7. The output of machine A feeds the input of B. This style of composition is called cascade composition or **serial composition**.

In the figure, output port o_1 from A feeds events to input port i_2 of B. Assume the data type of o_1 is V_1 (meaning that o_1 can take values from V_1 or be *absent*), and the

data type of i_2 is V_2. Then a requirement for this composition to be valid is that

$$V_1 \subseteq V_2 \ .$$

This asserts that any output produced by A on port o_1 is an acceptable input to B on port i_2. The composition **type check**s.

For cascade composition, if we wish the composition to be asynchronous, then we need to introduce some machinery for buffering the data that is sent from A to B. We defer discussion of such asynchronous composition to Chapter 6, where dataflow and process network models of computation will provide such asynchronous composition. In this chapter, we will only consider synchronous composition for cascade systems.

In synchronous composition of the cascade structure of Figure 5.7, a reaction of C consists of a reaction of both A and B, where A reacts first, produces its output (if any), and then B reacts. Logically, we view this as occurring in zero time, so the two reactions are in a sense **simultaneous and instantaneous**. But they are causally related in that the outputs of A can affect the behavior of B.

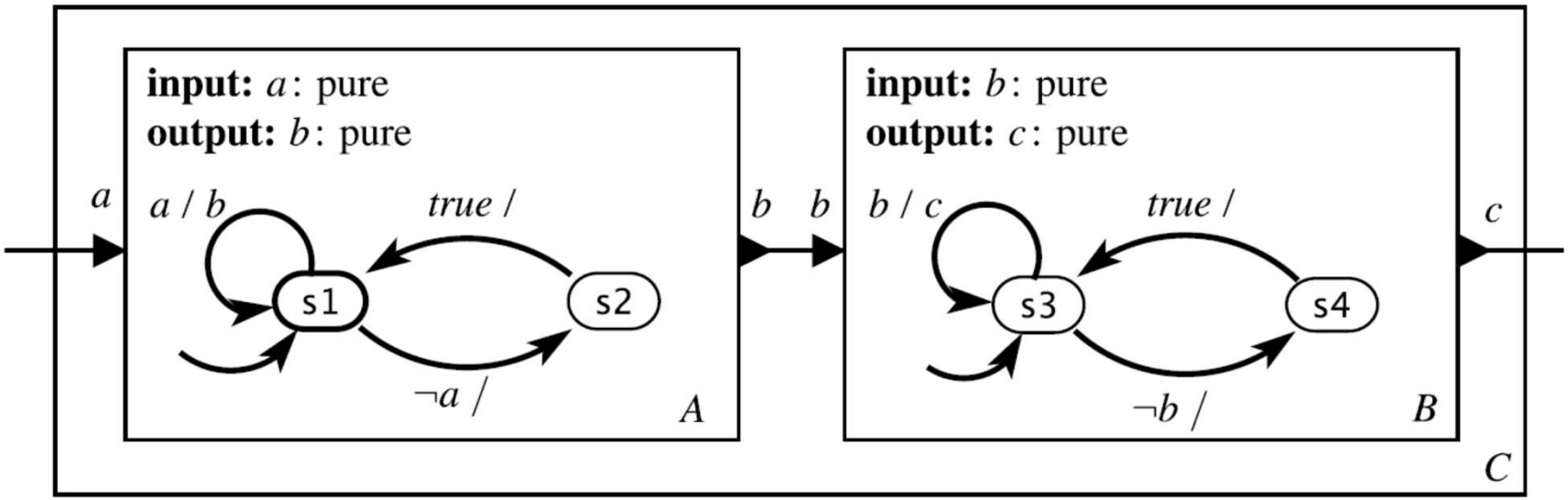

Figure 5.8: Example of a cascade composition of two FSMs.

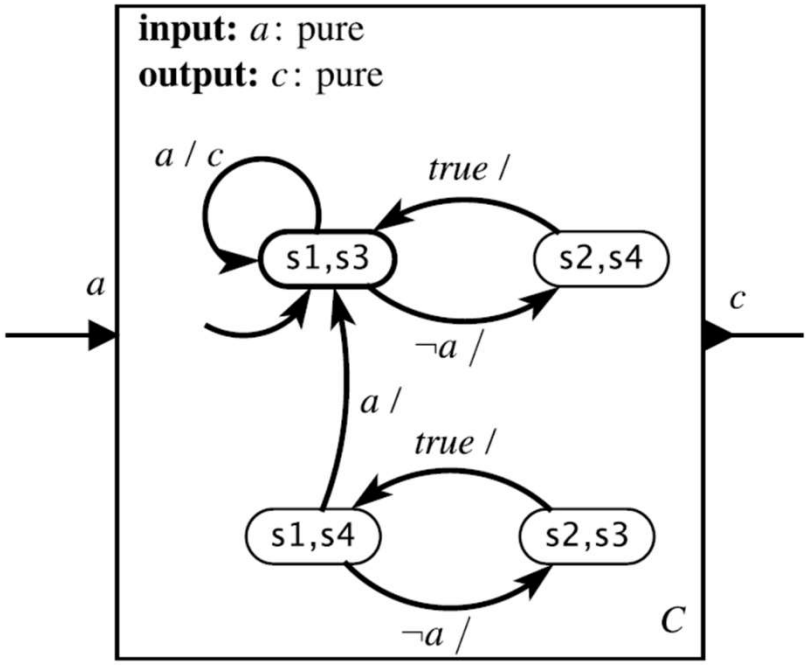

Figure 5.9: Semantics of the cascade composition of Figure 5.8, assuming synchronous composition.

Example 5.6: Consider the cascade composition of the two FSMs in Figure 5.8. Assuming synchronous semantics, the meaning of a reaction of C is given in Figure 5.9. That figure makes it clear that the reactions of the two machines are simultaneous and instantaneous. When moving from the initial state (s1, s3) to (s2, s4) (which occurs when the input a is absent), the composition machine C does not pass through (s2, s3)! In fact, (s2, s3) is not a reachable state! In this way, a *single* reaction of C encompasses a reaction of both A *and* B.

To construct the composition machine as in Figure 5.9, first form the state space as the cross product of the state spaces of the component machines, and then determine which transitions are taken under what conditions. It is important to remember that the transitions are simultaneous, even when one logically causes the other.

Example 5.7: Recall the traffic light model of Figure 3.10. Suppose that we wish to compose this with a model of a pedestrian crossing light, like that shown in Figure 5.10. The output *sigR* of the traffic light can provide the input *sigR* of the pedestrian light. Under synchronous cascade composition, the meaning of the composite is given in Figure 5.11. Note that unsafe states, such as (green, green), which is the state when both cars and pedestrians have a green light, are not reachable states, and hence are not shown.

In its simplest form, cascade composition implies an ordering of the reactions of the components. Since this ordering is well defined, we do not have as much difficulty with shared variables as we did with side-by-side composition. However, we will see that in more general compositions, the ordering is not so simple.

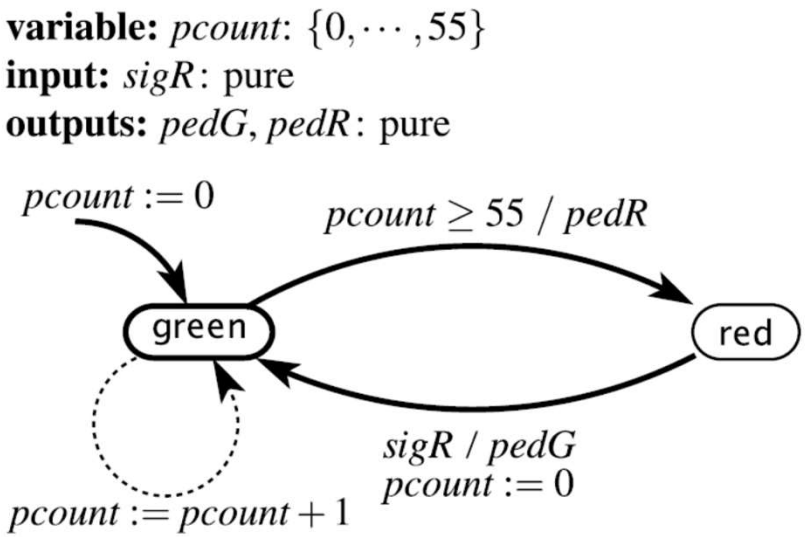

Figure 5.10: A model of a pedestrian crossing light, to be composed in a synchronous cascade composition with the traffic light model of Figure 3.10.

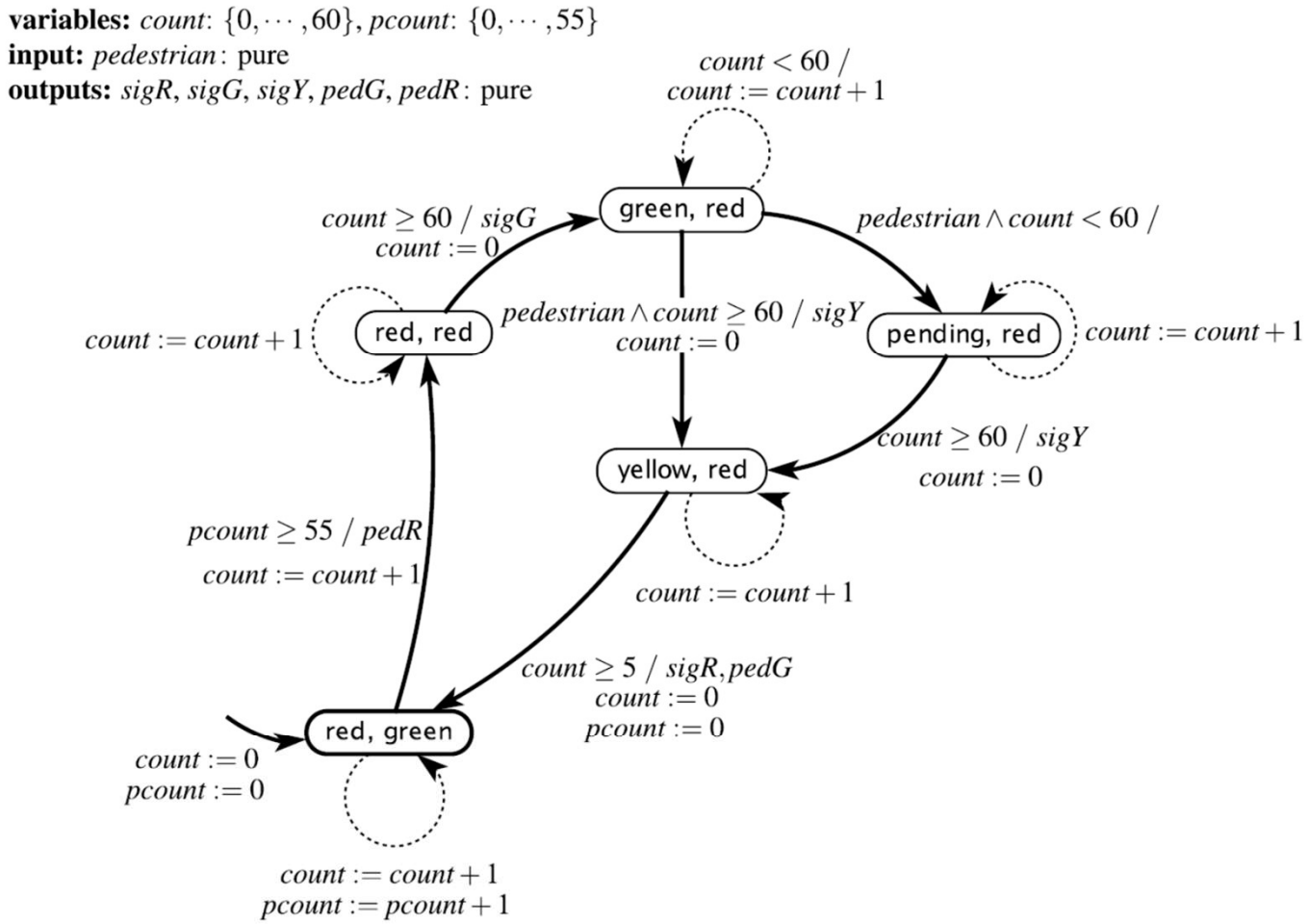

Figure 5.11: Semantics of a synchronous cascade composition of the traffic light model of Figure 3.10 with the pedestrian light model of Figure 5.10.

5.1.5 General Composition

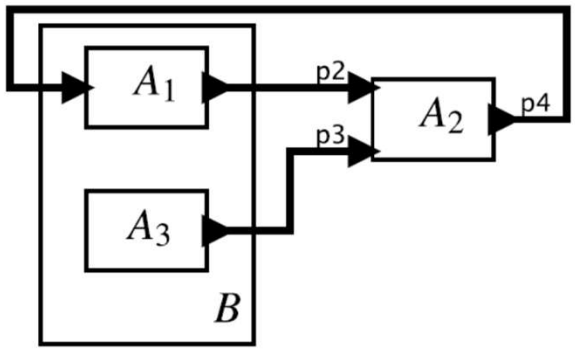

Figure 5.12: Arbitrary interconnections of state machines are combinations of side-by-side and cascade compositions, possibly creating cycles, as in this example.

Side-by-side and cascade composition provide the basic building blocks for building more complex compositions of machines. Consider for example the composition in Figure 5.12. A_1 and A_3 are a side-by-side composition that together define a machine B. B and A_2 are a cascade composition, with B feeding events to A_2. However, B and A_2 are also a cascade composition in the opposite order, with A_2 feeding events to B. Cycles like this are called feedback, and they introduce a conundrum; which machine should react first, B or A_2? This conundrum will be resolved in the next chapter when we explain the synchronous-reactive model of computation.

5.2 Hierarchical State Machines

In this section, we consider **hierarchical FSMs**, which date back to Statecharts (Harel, 1987). There are many variants of Statecharts, often with subtle semantic differences between them (von der Beeck, 1994). Here, we will focus on some of the simpler aspects only, and we will pick a particular semantic variant.

The key idea in hierarchical state machines is state refinement. In Figure 5.13, state B has a refinement that is another FSM with two states, C and D. What it means for the machine to be in state B is that it is in one of states C or D.

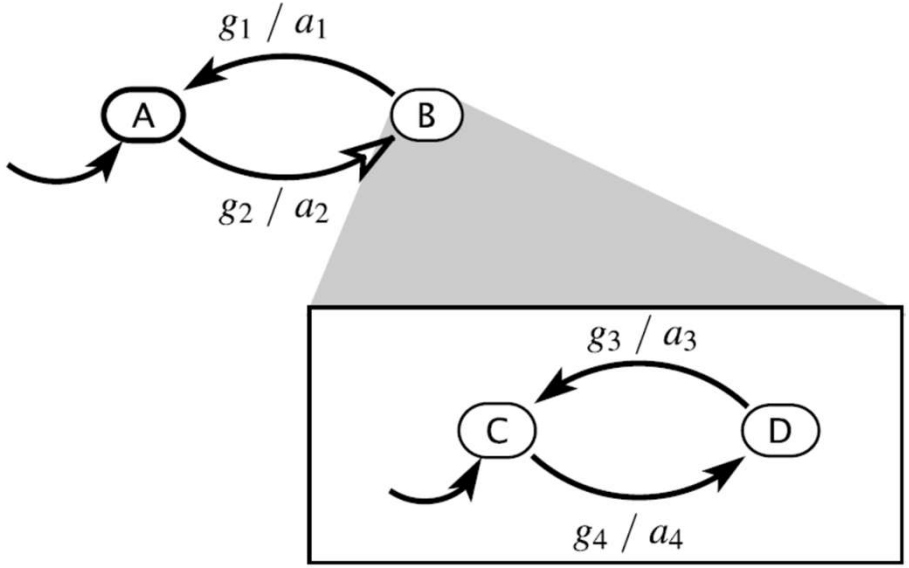

Figure 5.13: In a hierarchical FSM, a state may have a refinement that is another state machine.

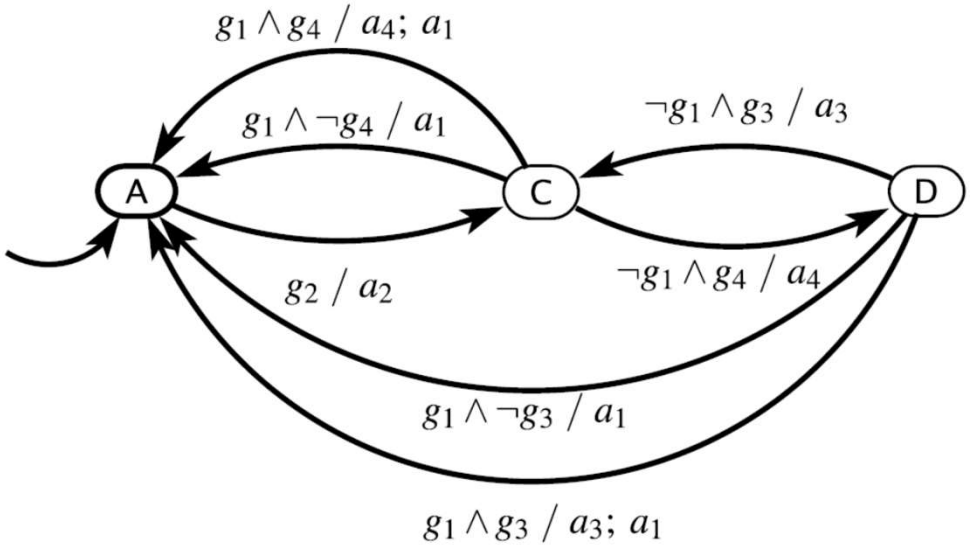

Figure 5.14: Semantics of the hierarchical FSM in Figure 5.13.

The meaning of the hierarchy in Figure 5.13 can be understood by comparing it to the equivalent flattened FSM in Figure 5.14. The machine starts in state A. When guard g_2 evaluates to true, the machine transitions to state B, which means a transition to state C, the initial state of the refinement. Upon taking this transition to C, the machine performs action a_2, which may produce an output event or set a variable (if this is an extended state machine).

There are then two ways to exit C. Either guard g_1 evaluates to true, in which case the machine exits B and returns to A, or guard g_4 evaluates to true and the machine transitions to D. A subtle question is what happens if both guards g_1 and g_4 evaluate to true. Different variants of Statecharts may make different choices at this point. It seems reasonable that the machine should end up in state A, but which of the actions should be performed, a_4, a_1, or both? Such subtle questions help account for the proliferation of different variants of Statecharts.

We choose a particular semantics that has attractive modularity properties (Lee and Tripakis, 2010). In this semantics, a reaction of a hierarchical FSM is defined in a depth-first fashion. The deepest refinement of the current state reacts first, then its container state machine, then its container, etc. In Figure 5.13, this means that if the machine is in state B (which means that it is in either C or D), then the refinement machine reacts first. If it is C, and guard g_4 is true, the transition is taken to D and action a_4 is performed. But then, as part of the same reaction, the top-level FSM reacts. If guard g_1 is also true, then the machine transitions to state A. It is important that logically these two transitions are simultaneous and instantaneous, so the machine does not actually go to state D. Nonetheless, action a_4 is performed, and so is action a_1. This combination corresponds to the topmost transition of Figure 5.14.

Another subtlety that arises is that if two actions are performed in the same reaction, they may conflict. For example, two actions may write different values to the same output port. Or they may set the same variable to different values. Our choice is that the actions are performed in sequence, as suggested by the semicolon in the action $a_4; a_1$. As in an imperative language like C, the semicolon denotes a sequence. As with an imperative language, if the two actions conflict, the later one dominates.

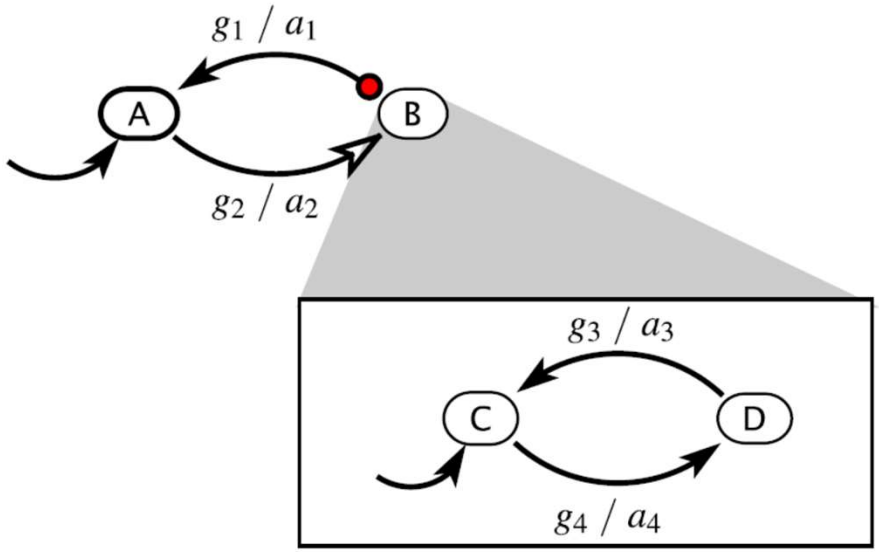

Figure 5.15: Variant of Figure 5.13 that uses a preemptive transition.

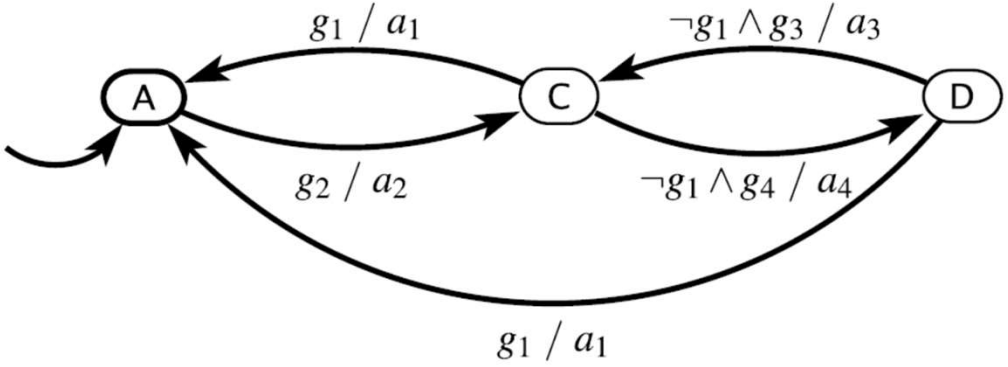

Figure 5.16: Semantics of Figure 5.15 with a preemptive transition.

Such subtleties can be avoided by using a **preemptive transition**, shown in Figure 5.15, which has the semantics shown in Figure 5.16. The guards of a preemptive transition are evaluated *before* the refinement reacts, and if any guard evaluates to true, the refinement does not react. As a consequence, if the machine is in state B and g_1 is true, then neither action a_3 nor a_4 is performed. A preemptive transition is shown with a (red) circle at the originating end of the transition.

Notice in Figures 5.13 and 5.14 that whenever the machine enters B, it always enters C, never D, even if it was previously in D when leaving B. The transition from A to B is called a **reset transition** because the destination refinement is reset to its initial state, regardless of where it had previously been. A reset transition is indicated in our notation with a hollow arrowhead at the destination end of a transition.

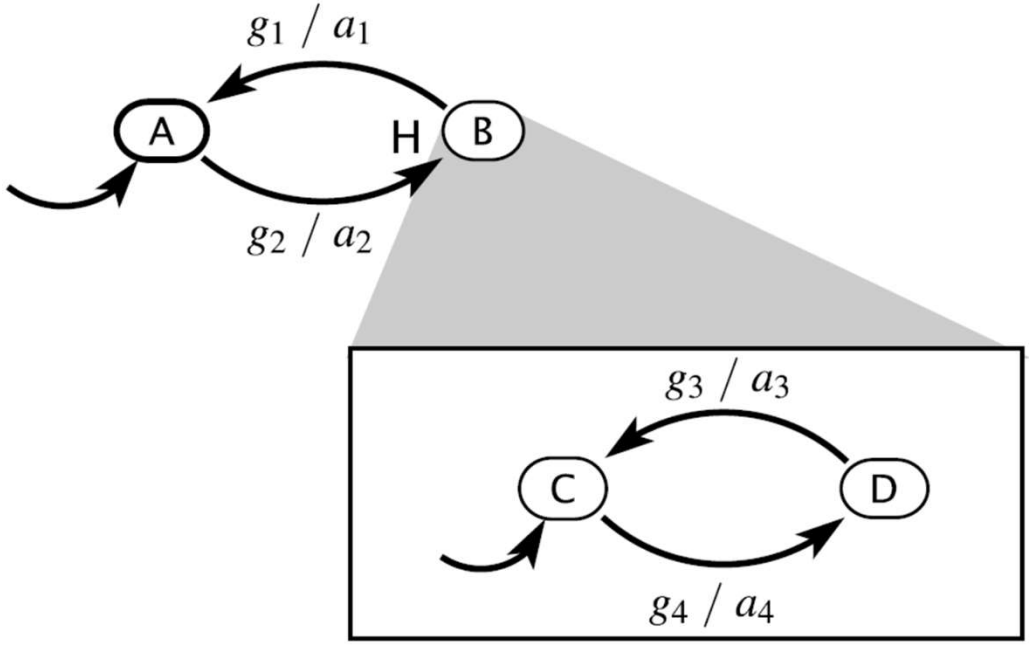

Figure 5.17: Variant of the hierarchical state machine of Figure 5.13 that has a history transition.

In Figure 5.17, the transition from A to B is a **history transition**, an alternative to a reset transition. In our notation, a solid arrowhead denotes a history transition. It may also be marked with an "H" for emphasis. When a history transition is taken, the destination refinement resumes in whatever state it was last in (or its initial state on the first entry).

The semantics of the history transition is shown in Figure 5.18. The initial state is labeled (A, C) to indicate that the machine is in state A, and if and when it next enters B it will go to C. The first time it goes to B, it will be in the state labeled (B, C) to indicate that it is in state B and, more specifically, C. If it then transitions to (B, D), and then back to A, it will end up in the state labeled (A, D), which means it is in state A, but if and when it next enters B it will go to D. That is, it remembers the history, specifically where it was when it left B.

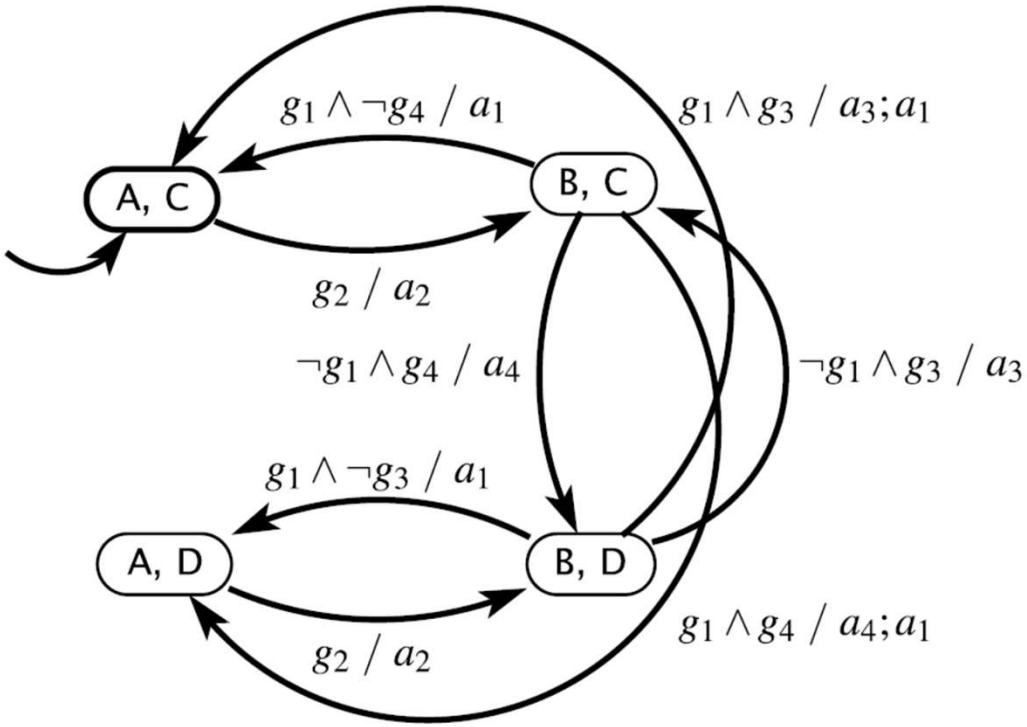

Figure 5.18: Semantics of the hierarchical state machine of Figure 5.17 that has a history transition.

As with concurrent composition, hierarchical state machines admit many possible meanings. The differences can be subtle. Considerable care is required to ensure that models are clear and that their semantics match what is being modeled.

5.3 Summary

Any well-engineered system is a composition of simpler components. In this chapter, we have considered two forms of composition of state machines, concurrent composition and hierarchical composition.

For concurrent composition, we introduced both synchronous and asynchronous composition, but did not complete the story. We have deferred dealing with feedback to the next chapter, because for synchronous composition, significant subtleties arise. For asynchronous composition, communication via ports requires additional mechanisms that are not (yet) part of our model of state machines. Even without communication via ports, significant subtleties arise because there are several possible semantics for asynchronous composition, and each has strengths and weaknesses. One choice of semantics may be suitable for one application and not for another. These subtleties motivate the topic of the next chapter, which provides more structure to concurrent composition and resolves most of these questions (in a variety of ways).

For hierarchical composition, we focus on a style originally introduced by Harel (1987) known as Statecharts. We specifically focus on the ability for states in an FSM to have refinements that are themselves state machines. The reactions of the refinement FSMs are composed with those of the machine that contains the refinements. As usual, there are many possible semantics.